

NLP Project 2026 WNLN Classification Arno LESAGE Minh QUAN HOANG

April 13, 2026

1 NLP Project 2026: Web Novel and Light Novel labels predictions

Authors: *Arno LESAGE (no. 22202985), Minh Quan Hoang (no. 22509118)*

README: How to use this notebook? To use this notebook, you may need a GPU with CUDA version 12.7, Python 3.12.0 and PyTorch 2.8+cu126. Since this material is not compatible with every desktop or laptop, you will find **Warning Sections** in this document, which may require the use of CUDA.

If you come across a **Warning Section**, you can run it if you have the compatible material and time/computing power to do so. Otherwise, it is advised that you skip the section entirely and go to the next **Pickle Restoration Point**, which will load the precomputed results obtained by these sections, stored in the `pickledFiles` folder. Note that the **Warning Sections** only contains computations, not visualisation or extensive interpretation. This means you can skip the computation parts without missing anything or getting stuck later on.

To run this notebook properly, it is advisable to use a Python environment and to install the correct version of the libraries mentioned in the `requirements.txt` file.

If any problems occur, you can refer to the PDF or HTML versions of this project. If a major problem occurs, please do not hesitate to contact the document's main author: Arno LESAGE.

1.1 1- Research Questions

Requirements (120 words): In the world of bookworms, it is very common to be able to identify the genre of a light novel (LN) or web novel (WN) based on its description. Unfortunately, this task is quite difficult for computers. Using the relatively unknown [Web/Light Novel Dataset](#), our goal is to predict the genre(s) of a book from its description. We hypothesise that a description reveals a particular semantic depending on the genres it is associated with, a method that has already been used for movie classification. For example, a LN with the **Horror** tag is more likely to have a gory description than a **Romance/School Life** LN. Additionally, we hypothesise a high correlation between genres (e.g. **Adventure** implies **Fantasy** or **Drama/Romance**).

Required Reflection:

[Q1] Why is your research question non trivial? There are multiple difficulties with this research question, but the two main ones are the large number of genres (35 in our dataset) and their correlation, which would make attempts at discrimination difficult. (Note: perhaps this correlation

could be turned into a strength.) Another potential difficulty with our dataset is the imbalance in label distribution, which will be confirmed or not during experimentation.

[Q2] What linguistic phenomenon does it involve? The linguistic phenomenon we want to capture here is the underlying semantics of different genres. We expect genres such as *Romance/Comedy* to have more joyful words and sentence structures, genres such as *Adult/Ecchi/Harem/Gender Bender* to have more tendentious ones, and genres such as *Shoujo/Shounen/Adventure* to have more youthful ones, and so on.

[Q3] What would falsify your hypothesis? One possible reason why our hypothesis could be falsified is that translating the description from the original language (Chinese, Korean, Japanese or Vietnamese) into English may have removed some details and distorted the underlying semantics. We could also consider the type of description: some description may not consider the story itself, but rather how it was produced, by whom and so on.

1.2 2.1 - Dataset Selection

The dataset chosen for this project is the [Web/Light Novel Dataset](#) (original-wn-dataset.csv).

- **Source:** Kaggle, Web/Light Novel Dataset, Prem Shinde, 2024
- **Domain:** Literature, Web/Light Novels, Asian Literature
- **Label Distribution:** Multilabel, Heterogeneous (some labels have 106, others 7523), “exponentially” unbalanced distributions of labels.
- **Potential biases:** Correlation between books origins and labels (China has more chance to produce *Wuxia* or *Xuanhuan* for example), correlation between labels, traduction failures of Chinese, Japanese, Korean, Vietnamese, ... to English.
- **Ethical Consideration:** Methodology for extraction is unknown. Apart from this, this dataset does not exhibit particular ethical considerations.
- **Size:** 11770 labeled documents, 74.87MB.
- **Main Content:** Title, Description [Classifiers], Source (URL), Genres [Predicted variable], Tags (Sub genres), related series, authors, artists, original language...

Despite the relatively poor documentation, this dataset was chosen because it is more complete (with more instances) than other available alternatives, and has a greater variety of genres and tags. It is also easier to navigate. Alternatives could have included: - [RanobeDB](#): SQL Database containing series and books for manga, LN and WN. (difficult to navigate), - [MyAnimeList Dataset](#): Very complete and good documentation, but has less variety and may not contains LN and WN independent productions. - [Anime Title Dataset](#): Complete, but has very low number of instances (<1000) and Anime Only.

More informations on the shape of the dataset in the part **(2.1 - In Depth)**

1.2.1 Libraries

```
[1]: # Default package
import re
from collections import Counter
from itertools import chain
import time
import pickle as pkl
```

```

# Visualisation
%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import seaborn as sns
import networkx as nx

# Base ML
import numpy as np
import pandas as pd
import csv

# NN
import torch
import torch.nn as nn
torch.cuda.init()
from transformers import BertTokenizer, BertModel
print(torch.cuda.is_available())

# Preprocessing
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
import spacy
nlp = spacy.load("en_core_web_sm")

# Vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

# Multilabel ML
from skmultilearn.model_selection import iterative_train_test_split
from sklearn.preprocessing import MultiLabelBinarizer
from skmultilearn.problem_transform import BinaryRelevance
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, accuracy_score, recall_score, precision_score, hamming_loss, jaccard_score
from sklearn.manifold import TSNE

```

True

1.2.2 Import the dataset (original-wn)

First let's expose the columns:

```

[2]: df = pd.read_csv("datasets/original-wn-dataset.csv")
df.describe(include="all")

```

```

[2]:
      novel_id      url \
count  11770.000000      11770
unique      NaN      11770
top      NaN  after-the-breakup-i-went-viral-in-the-entertai...
freq      NaN      1
mean   34047.713084      NaN
std    19129.992708      NaN
min      10.000000      NaN
25%    18403.250000      NaN
50%    35094.000000      NaN
75%    50761.000000      NaN
max     65378.000000      NaN

      title associated_names \
count      11770      11770
unique      11768      11619
top  The Imperial Concubine Just Wants To Be a Salt...      []
freq      2      97
mean      NaN      NaN
std      NaN      NaN
min      NaN      NaN
25%      NaN      NaN
50%      NaN      NaN
75%      NaN      NaN
max      NaN      NaN

      img_url      showtype \
count      11770      11700
unique      10159      20
top  https://www.novelupdates.com/img/noimagefound.jpg  Web Novel (CN)
freq      1607      5592
mean      NaN      NaN
std      NaN      NaN
min      NaN      NaN
25%      NaN      NaN
50%      NaN      NaN
75%      NaN      NaN
max      NaN      NaN

      genres  tags description \
count      11770  11770      11766
unique      6014  11215      11676
top  ['Fantasy' 'Romance']      []  N/A\n
freq      230      155      66
mean      NaN      NaN      NaN
std      NaN      NaN      NaN
min      NaN      NaN      NaN

```

25%	NaN	NaN	NaN
50%	NaN	NaN	NaN
75%	NaN	NaN	NaN
max	NaN	NaN	NaN

	related_series	...	publishers	\
count	11770	...	11770	
unique	1325	...	1213	
top	{ 'related_series': array([], dtype=object), 't... ... ['jjwxc']			
freq	10398	...	2216	
mean	NaN	...	NaN	
std	NaN	...	NaN	
min	NaN	...	NaN	
25%	NaN	...	NaN	
50%	NaN	...	NaN	
75%	NaN	...	NaN	
max	NaN	...	NaN	

	en_pubs	release_frequency	weekly_rank	monthly_rank	all_time_rank	\
count	11770	11769	11770	11770	11770	
unique	69	3721	474	2600	8282	
top	[]	Every 2000 Day(s)	#2000	#2000	#14613	
freq	10454	118	6097	5699	13	
mean	NaN	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	NaN	

	monthly_rank_reading_list	all_time_rank_reading_list	\
count	11770	11770	
unique	3091	6533	
top	#15649	#11754	
freq	418	13	
mean	NaN	NaN	
std	NaN	NaN	
min	NaN	NaN	
25%	NaN	NaN	
50%	NaN	NaN	
75%	NaN	NaN	
max	NaN	NaN	

	total_reading_list_rank	\
count	11770.000000	
unique	NaN	

```

top          NaN
freq         NaN
mean        1664.066950
std         2436.728268
min          16.000000
25%         300.000000
50%         740.000000
75%        1981.000000
max        29460.000000

```

```

                                chapters
count                        11770
unique                      10488
top    {'chapters': array([], dtype=object), 'total': 0}
freq                        1283
mean                        NaN
std                        NaN
min                        NaN
25%                        NaN
50%                        NaN
75%                        NaN
max                        NaN

```

[11 rows x 30 columns]

For our NLP classification task, we are interested on the following columns `novel_id`, `title`, `genres`, `tags` and `description`.

```
[3]: df_filtered = df.loc[:, ["novel_id", "title", "genres", "tags", "description"]]
df_filtered.head()
```

```
[3]:  novel_id                                title \
0      64494  After the Breakup, I Went Viral in the Enterta...
1      43962                                Mahou Sekai no Monogatari
2      25506                                I am a Good Man
3      50373                                Scum Girl Practice Manual
4      48291  Unable to Divorce After Signing the Marriage C...
```

```

                                genres \
0      ['Drama' 'Fantasy' 'Romance' 'Supernatural']
1      ['Fantasy' 'Romance' 'Shoujo']
2      ['Action' 'Adventure' 'Fantasy' 'Historical' '...'
3      ['Romance' 'School Life' 'Shoujo']
4      ['Psychological' 'Romance' 'Shounen Ai' 'Slice...
```

```

                                tags \
0      ['Amnesia' 'Beautiful Female Lead' 'Clever Pro...
1      ['Adventurers' 'Elemental Magic' 'Familiars' '...
```

```

2 ['Beautiful Female Lead' 'Businessmen' 'Childc...
3 ['Female Protagonist' 'Modern Day' 'Short Stor...
4 ['Arranged Marriage' 'Calm Protagonist' 'Cold ...

                                description
0 Luo Ning was a cannon fodder, a female support...
1 A world where magic exists.\nThis is the story...
2 To sum up in a sentence, transmigrating to dif...
3           My goal is, to be a scum woman.\n
4 Lin You He excelled in everything - his looks...

```

1.2.3 Preprocess Dataset Format

Using the `.head()` command, we observe that the `genres` and `tags` columns are encoded into space separated list, which are not recognised in Python, so we will have to change the format to proper lists.

```

[4]: def colToList (x:str) -> list[str]:
      # Detect the separator
      x = re.sub(r"[\n ].?", "", x)

      # Remove the indicator of the raw space separated list and split on coma
      xSplit = x.removeprefix("[").removesuffix("]").split(",")

      # Return the list where the text is capitalize (it is just prettier)
      return list(map(str.capitalize, xSplit))

# Apply the split
df_filtered.loc[:, "genres"] = df_filtered.loc[:, "genres"].apply(lambda x:
    ↪ colToList(x))
df_filtered.loc[:, "tags"] = df_filtered.loc[:, "tags"].apply(lambda x:
    ↪ colToList(x))
df_filtered.head()

```

```

[4]:   novel_id                                title \
0      64494  After the Breakup, I Went Viral in the Enterta...
1      43962                                Mahou Sekai no Monogatari
2      25506                                I am a Good Man
3      50373                                Scum Girl Practice Manual
4      48291  Unable to Divorce After Signing the Marriage C...

                                genres \
0      [Drama, Fantasy, Romance, Supernatural]
1      [Fantasy, Romance, Shoujo]
2  [Action, Adventure, Fantasy, Historical, Roman...
3      [Romance, School life, Shoujo]
4  [Psychological, Romance, Shounen ai, Slice of ...

```

```

                                tags \
0  [Amnesia, Beautiful female lead, Clever protagon...
1  [Adventurers, Elemental magic, Familiars, Fema...
2  [Beautiful female lead, Businessmen, Childcare...
3  [Female protagonist, Modern day, Short story, ...
4  [Arranged marriage, Calm protagonist, Cold pro...

                                description
0  Luo Ning was a cannon fodder, a female support...
1  A world where magic exists.\nThis is the story...
2  To sum up in a sentence, transmigrating to dif...
3                                My goal is, to be a scum woman.\n
4  Lin You He excelled in everything - his looks...

```

1.3 2.1bis (and 3.1.) - in depth - Some stats on the dataset

Now that everything has been imported correctly, we will analyse some statistics from our dataset to gain an understanding of it. First, we will try to determine the language (domain, slang, ...) used in our data. ##### **Language description** To achieve this goal, we will first examine the most frequent words in our dataset. Then, we will build a function to compare the dataset with and without stop words.

```

[5]: def plotDescriptionLanguage(df:pd.DataFrame, col="description",
    ↪stopFiltered=True):
    descriptionList = list(map(str,df.loc[:,col].to_list()))
    descriptionString = " ".join(descriptionList)
    # Match all words and digits
    wordDescriptionList = re.findall(r"[\w\d-]+",descriptionString)

    # Remove the stopwords if asked
    wordDescriptionList = [i.capitalize() for i in wordDescriptionList if (i.
    ↪lower() not in stop_words) or not stopFiltered]

    # Apply the counter
    wordDescriptionListCounter = Counter(wordDescriptionList)

    # Sort the counter
    wordDescriptionListCounterSorted = [],[]
    for word in sorted(wordDescriptionListCounter, key=lambda x:
    ↪wordDescriptionListCounter[x], reverse=True):
        wordDescriptionListCounterSorted[0].append(word)
        wordDescriptionListCounterSorted[1].
    ↪append(wordDescriptionListCounter[word])

    # Make a plot
    plt.close("all")

```



```

sns.set_style("dark")
sns.barplot(
    x=wordDescriptionListCounterSorted[1][:40],
    y=wordDescriptionListCounterSorted[0][:40],
    palette="BrBG"
)
plt.title(f"Wordcount {'(stopwords filtered)' if stopFiltered else ''}")
plt.xlabel("Wordcount")
plt.ylabel("Words")

plt.show()

return wordDescriptionListCounterSorted

```

Now that we have the function, we can start without stopwords filtering:

```

[6]: df_filteredWordCounter = plotDescriptionLanguage(df_filtered,
↪stopFiltered=False)

```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\1773452185.py:22:

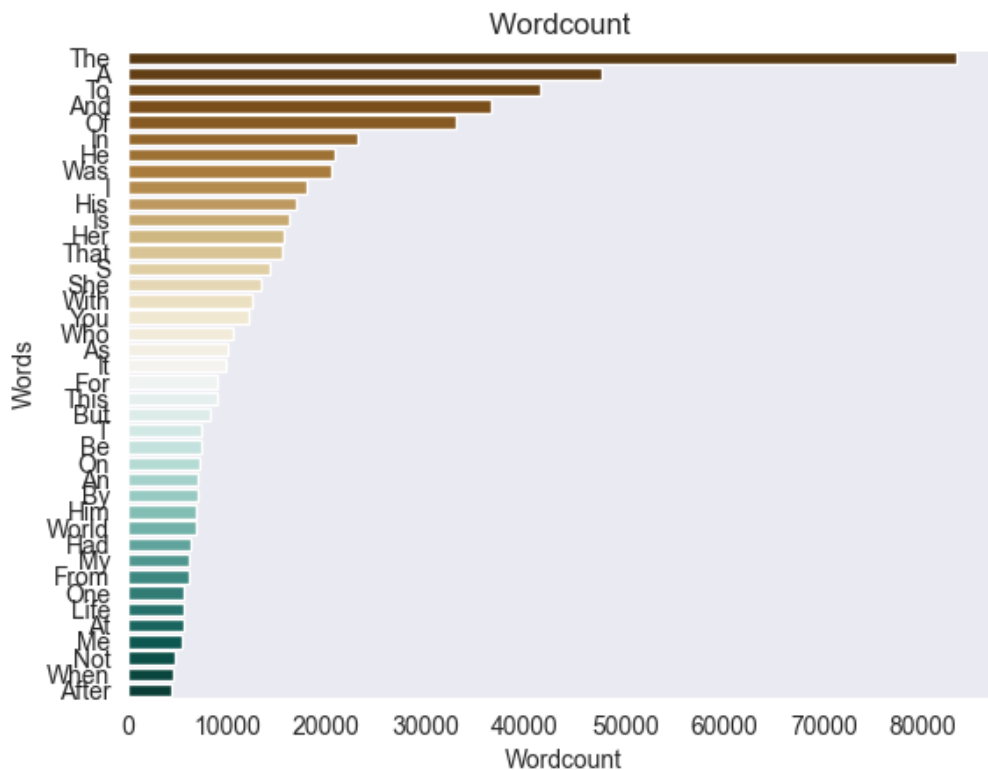
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(

```



As expected, we quickly observe that all of the most frequent words are stop words (a, the, to, and, of, etc.). This is bad news. In this context, these words will not exhibit high semantic importance in the document, but they could possess tremendous weight in our future embeddings. We will now produce the same plot without stopwords, using the English stopwords list from **nltk**.

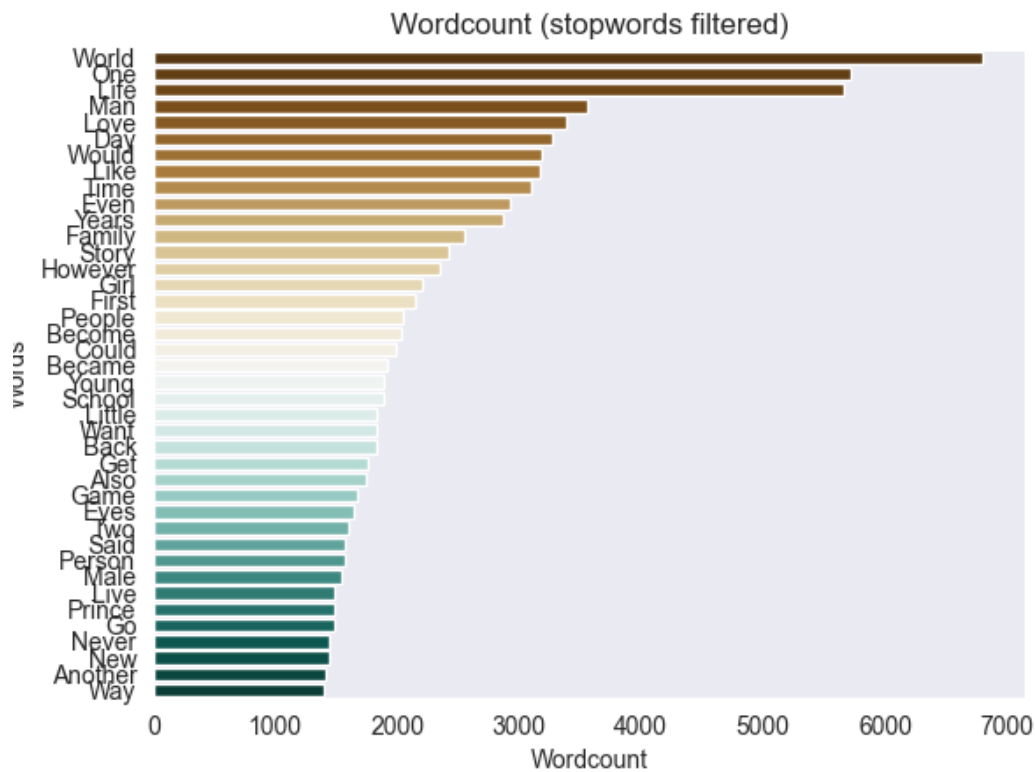
```
[7]: _ = plotDescriptionLanguage(df_filtered)
```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\1773452185.py:22:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



Here, the difference is clear. Setting aside generic stopwords, we see some interesting words such as *world*, *life*, *prince*, *story* and *time*, which could be tied to genres such as **Fantasy** or **Adventure** given their popularity. Another predominant family emerges here with *School*, *Young*, *Family* and *Love*, which are typical of the genres **School Life** and **Romance**. Ultimately, we see some “specialised” topics compared to the political, economic and scientific texts, or newspaper articles, that we are used to.

But what about the least used words and their frequency globally?

```
[8]: df_filteredWordDescription = pd.DataFrame(columns=["Word", "Frequency"])
df_filteredWordDescription["Word"] = pd.Series(df_filteredWordCounter[0])
df_filteredWordDescription["Frequency"] = pd.Series(df_filteredWordCounter[1])
df_filteredWordDescription
```

```
[8]:
```

	Word	Frequency
0	The	83384
1	A	47692
2	To	41495
3	And	36481
4	Of	33092
...	...	...

41020	Strength-worshiping	1
41021	Adult-based	1
41022	Seryu	1
41023	Wisemen	1
41024	Overkill	1

[41025 rows x 2 columns]

Using the table, we can see that the least used words can broadly be categorised into four groups: compound words (e.g. *Strength-worshiping*, *Adult-based*, *Good-nature*), proper names (e.g. *Tsuruki*, *Zhaoying*, *Ikaros*), specialised terms (e.g. *Euthanasia*, *Canines*, *Serializing*), and non-English terms (e.g. *Otouto*, *Giraffe*, *100g*, *1920*,).

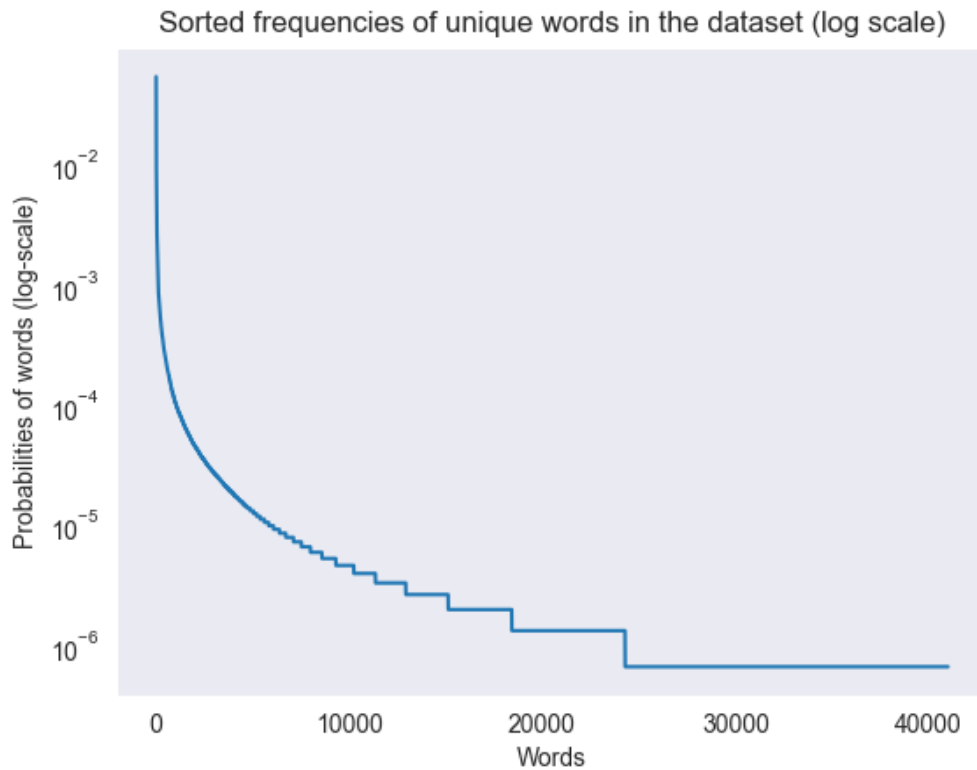
Additionally, we observe that 22,603 unique words (55.10%) appear fewer than two times, and that 16,732 words (40.78%) appear only once. This observation makes it necessary to filter out rare words.

Ultimately, only approximately 12.3% of words could be considered “frequent” (20+ occurrences). Only 3.64% of “very frequent words” (100+ occurrences) can be considered as such. This clearly shows the concentration of occurrences and the heterogeneity of the dictionary used.

Let us examine this proposition in more detail:

```
[9]: # Make a plot
plt.close("all")
sns.set_style("dark")
sns.lineplot(df_filteredWordDescription["Frequency"] /
             ↪df_filteredWordDescription["Frequency"].sum())
plt.yscale("log")

plt.title(f"Sorted frequencies of unique words in the dataset (log scale)")
plt.xlabel("Words")
plt.ylabel("Probabilities of words (log-scale)")
plt.show()
```



This graph (log-linear scale) clearly shows that the proportion of word occurrence drops quickly after the first ten words, and that after a thousand words, the proportion drops below $10E-5$. Ultimately, this plot confirms what we observed in the above table: In most cases, a word is very rare, appearing fewer than ten times over 75% of the time.

Genres stats First how genres are distributed:

```
[10]: # Get a flatten list of genre
genresList = list(chain.from_iterable(df_filtered.loc[:, "genres"].tolist()))

# Make a counter and sort it
genresCounter = Counter(genresList)
genresCounterSorted = [], []
for genre in sorted(genresCounter, key=lambda x: genresCounter[x], reverse=True):
    genresCounterSorted[0].append(genre)
    genresCounterSorted[1].append(genresCounter[genre])

print(genresCounterSorted)
# Make a plot
plt.close("all")
sns.set_style("dark")
```

```
sns.barplot(
    x=genresCounterSorted[1],
    y=genresCounterSorted[0],
    palette="BrBG"
)
plt.title("Genres frequencies")
plt.xlabel("Frequencies")
plt.ylabel("Genres")

plt.show()
```

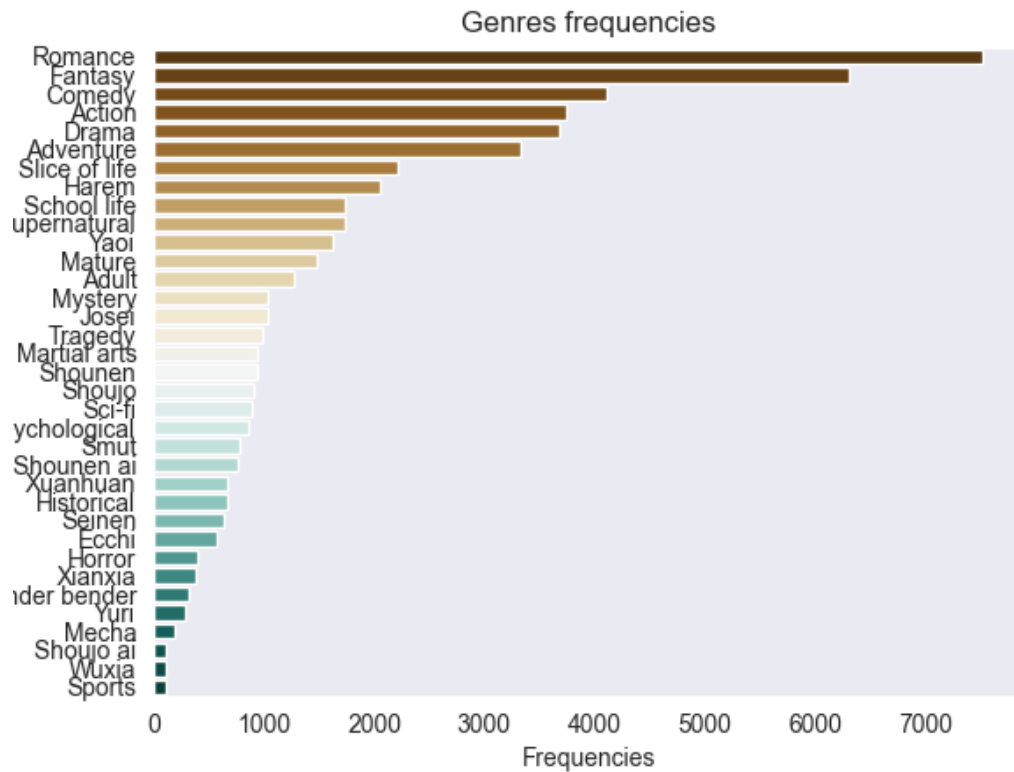
```
[['Romance', 'Fantasy', 'Comedy', 'Action', 'Drama', 'Adventure', 'Slice of
life', 'Harem', 'School life', 'Supernatural', 'Yaoi', 'Mature', 'Adult',
'Mystery', 'Josei', 'Tragedy', 'Martial arts', 'Shounen', 'Shoujo', 'Sci-fi',
'Psychological', 'Smut', 'Shounen ai', 'Xuanhuan', 'Historical', 'Seinen',
'Ecchi', 'Horror', 'Xianxia', 'Gender bender', 'Yuri', 'Mecha', 'Shoujo ai',
'Wuxia', 'Sports'], [7523, 6309, 4105, 3743, 3672, 3332, 2210, 2048, 1743, 1731,
1627, 1486, 1277, 1043, 1042, 984, 941, 935, 903, 886, 865, 784, 769, 673, 669,
639, 580, 406, 377, 313, 294, 184, 118, 112, 106]]
```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\3904004382.py:15:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



```
[11]: print(genresCounterSorted)
```

```
[['Romance', 'Fantasy', 'Comedy', 'Action', 'Drama', 'Adventure', 'Slice of
life', 'Harem', 'School life', 'Supernatural', 'Yaoi', 'Mature', 'Adult',
'Mystery', 'Josei', 'Tragedy', 'Martial arts', 'Shounen', 'Shoujo', 'Sci-fi',
'Psychological', 'Smut', 'Shounen ai', 'Xuanhuan', 'Historical', 'Seinen',
'Ecchi', 'Horror', 'Xianxia', 'Gender bender', 'Yuri', 'Mecha', 'Shoujo ai',
'Wuxia', 'Sports'], [7523, 6309, 4105, 3743, 3672, 3332, 2210, 2048, 1743, 1731,
1627, 1486, 1277, 1043, 1042, 984, 941, 935, 903, 886, 865, 784, 769, 673, 669,
639, 580, 406, 377, 313, 294, 184, 118, 112, 106]]
```

We can see here that some genres are really underrepresented. This is the case for **Sport**, **Gender Bender**, **Wuxia** and **Mecha**. Consequently, we will remove the labels from the dataset to facilitate our prediction.

Additionally, in this context, some tags are very similar and could be interchangeable. For example, the tags **Shounen Ai** and **Yaoi** are defined in the same way, but the former focuses on relationships between boys and the latter on more explicit activities (I don't read such things, so I could be mistaken, but this is what I found after some online research). With different semantics, the same interchangeability remarks can be made for the following genres:

- Smut vs Adult,
- Xuanhuan vs Xianxia,

- Yuri vs Shoujo Ai,
- Yaoi vs Shounen Ai.

We will simply merge the concerned genres.

```
[12]: # Dictionary of interchangeable genres
transferGenres = {
    "Smut": "Adult",
    "Shoujo ai": "Yuri",
    "Xianxia": "Xuanhuan",
    "Shounen ai": "Yaoi"
}

# Genres to remove
forbiddenGenres = set([
    "Sports",
    "Wuxia",
    "Mecha",
    "Gender bender"
])

# For each document
dfGenresFiltered = []
for genres in df_filtered.loc[:, "genres"]:
    genres_copy: list = genres.copy()

    # Changes the genres by removing/transforming the labels
    for genre in genres:
        if genre in forbiddenGenres:
            genres_copy.remove(genre)
        if genre in transferGenres:
            genres_copy.remove(genre)
            genres_copy.append(transferGenres[genre])
    dfGenresFiltered.append(genres_copy)

df_genresFiltered = df_filtered.copy(deep=True)
df_genresFiltered.loc[:, "genres"] = pd.Series(dfGenresFiltered)
```

What does it looks like after transformation?

```
[13]: # Get a flatten list of genre
genresList = list(chain.from_iterable(df_genresFiltered.loc[:, "genres"].
    ↪tolist()))

# Make a counter and sort it
genresCounter = Counter(genresList)
genresCounterSorted = [], []
for genre in sorted(genresCounter, key=lambda x: genresCounter[x], reverse=True):
    genresCounterSorted[0].append(genre)
```



```

genresCounterSorted[1].append(genresCounter[genre])

print(genresCounterSorted)
# Make a plot
plt.close("all")
sns.set_style("dark")
sns.barplot(
    x=genresCounterSorted[1],
    y=genresCounterSorted[0],
    palette="BrBG"
)
plt.title("Genres frequencies")
plt.xlabel("Frequencies")
plt.ylabel("Genres")

plt.show()

```

```

[['Romance', 'Fantasy', 'Comedy', 'Action', 'Drama', 'Adventure', 'Yaoi', 'Slice
of life', 'Adult', 'Harem', 'School life', 'Supernatural', 'Mature', 'Xuanhuan',
'Mystery', 'Josei', 'Tragedy', 'Martial arts', 'Shounen', 'Shoujo', 'Sci-fi',
'Psychological', 'Historical', 'Seinen', 'Ecchi', 'Yuri', 'Horror'], [7523,
6309, 4105, 3743, 3672, 3332, 2396, 2210, 2061, 2048, 1743, 1731, 1486, 1050,
1043, 1042, 984, 941, 935, 903, 886, 865, 669, 639, 580, 412, 406]]

```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\3172256800.py:15:

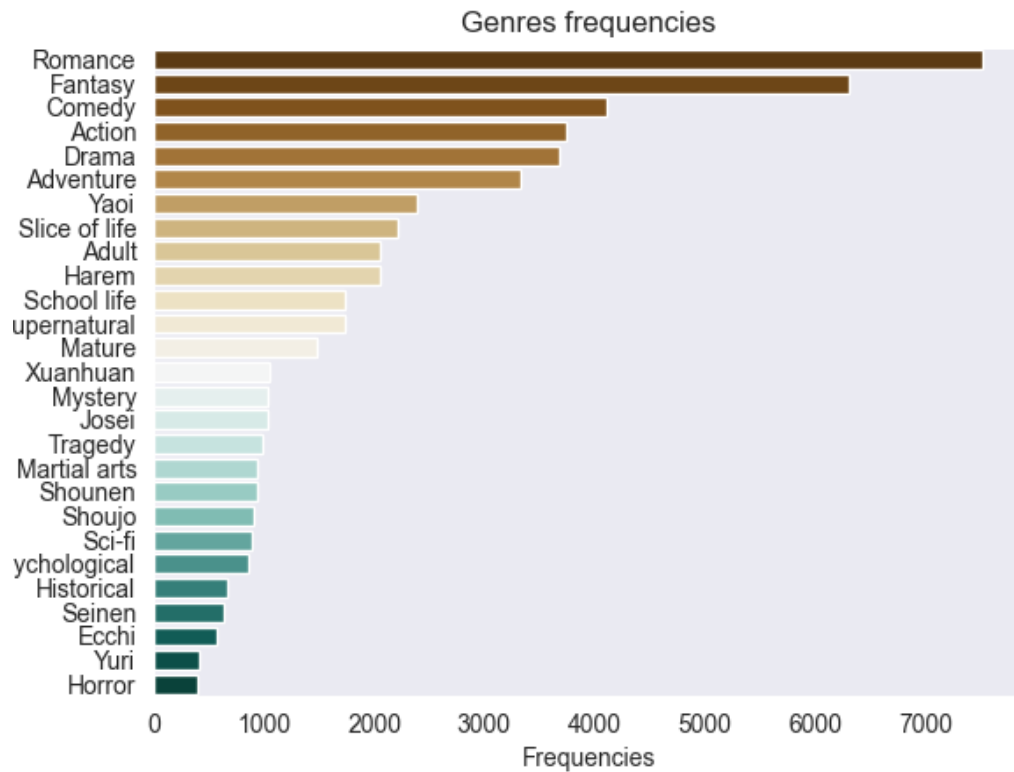
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(

```



It turned good. Second, how do genres interact together:

```
[14]: # Create the genre id dictionary
genreToGenreID = {genre:label for label, genre in enumerate(set(genresList))}
genreIDtoGenre = {value:key for key, value in genreToGenreID.items()}
print(genreToGenreID, genreIDtoGenre, sep="\n")

# Create the genreCount dictionary
genreMatrix = np.zeros((len(genreIDtoGenre),len(genreIDtoGenre)), dtype=int)

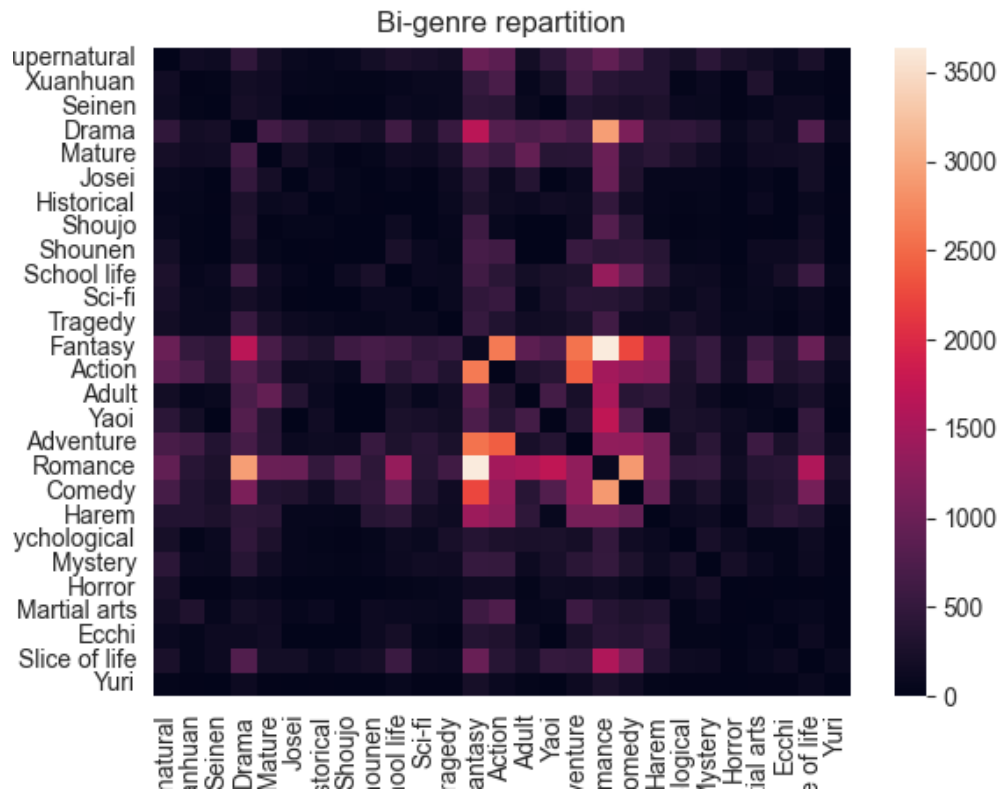
## For all genres
for genres in df_genresFiltered.loc[:, "genres"].tolist():
    # If it is the only genre, then loop to itself
    if len(genres)==1:
        genreMatrix[genreToGenreID[genres[0]], genreToGenreID[genres[0]]] += 1
        continue
    # Otherwise, add a link to all couple of genre which are different
    for genre1 in genres:
        for genre2 in genres:
            if genre1 != genre2:
                genreMatrix[genreToGenreID[genre1], genreToGenreID[genre2]] += 1
```

```
#print(genreMatrix)
```

```
{'Supernatural': 0, 'Xuanhuan': 1, 'Seinen': 2, 'Drama': 3, 'Mature': 4,
'Josei': 5, 'Historical': 6, 'Shoujo': 7, 'Shounen': 8, 'School life': 9, 'Sci-
fi': 10, 'Tragedy': 11, 'Fantasy': 12, 'Action': 13, 'Adult': 14, 'Yaoi': 15,
'Adventure': 16, 'Romance': 17, 'Comedy': 18, 'Harem': 19, 'Psychological': 20,
'Mystery': 21, 'Horror': 22, 'Martial arts': 23, 'Ecchi': 24, 'Slice of life':
25, 'Yuri': 26}
{0: 'Supernatural', 1: 'Xuanhuan', 2: 'Seinen', 3: 'Drama', 4: 'Mature', 5:
'Josei', 6: 'Historical', 7: 'Shoujo', 8: 'Shounen', 9: 'School life', 10: 'Sci-
fi', 11: 'Tragedy', 12: 'Fantasy', 13: 'Action', 14: 'Adult', 15: 'Yaoi', 16:
'Adventure', 17: 'Romance', 18: 'Comedy', 19: 'Harem', 20: 'Psychological', 21:
'Mystery', 22: 'Horror', 23: 'Martial arts', 24: 'Ecchi', 25: 'Slice of life',
26: 'Yuri'}
```

```
[15]: # Make a plot
plt.close("all")
sns.heatmap(genreMatrix,
            xticklabels=list(genreToGenreID.keys()),
            yticklabels=list(genreToGenreID.keys()))
plt.title("Bi-genre repartition")
plt.xlabel("Genre")
plt.ylabel("Genre")

plt.show()
```



```
[16]: # Make the graph for visibility
G = nx.from_numpy_array(genreMatrix)
G = nx.relabel_nodes(G, dict(enumerate(list(genreToGenreID.keys()))))

# Compute the weights of each edges for the graph
weights = np.array(list(nx.get_edge_attributes(G,"weight").values()))
weights = weights/weights.max()

# Compute the degree of each node to infer nodes sizes
deg = dict(G.degree(weight="weight"))
node_sizes = [100 + 400 * deg[n] / max(deg.values()) for n in G.nodes()]

# Plot the graph
plt.close("all")
nx.draw_networkx(
    G,                                     # The graph
    pos=nx.shell_layout(G),               # The layout
    width=3*weights,                      # The width of the edges
    edge_color=weights,                   # Edge colour defined by the weights
    edge_cmap=plt.cm.GnBu,               # Colour map of the edges
```

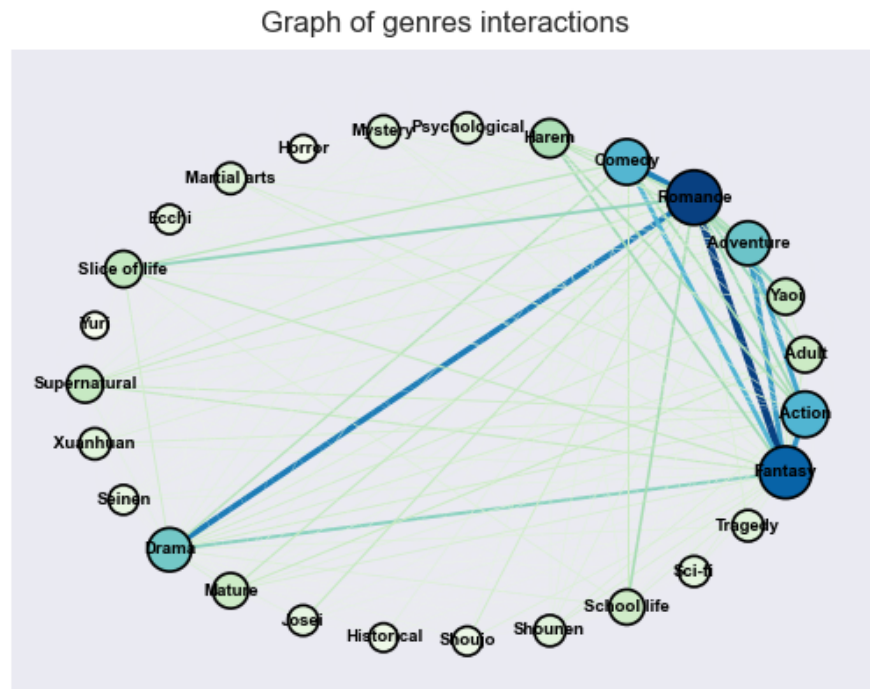
```

font_size="x-small",      # The font size
font_color="black",       # The font colour
font_weight="bold",      # We want bold
font_family="Arial",     # We want Arial
node_size=node_sizes,    # The size of the nodes
edgecolors="black",      # The border colour of the nodes
linewidths=1.2,          # Contour colour of the nodes
node_color=node_sizes,   # Colour of the node depends on the
↪degree

cmap=plt.cm.GnBu         # Colour map of the nodes
)

# Show the graph
plt.title("Graph of genres interactions")
plt.show()

```



Here, we observe a high degree of correlation between genres, which will either hinder or enhance our work depending on our classifications. For example, the triad **Adventure**, **Fantasy** and **Action** frequently appears, so if we have the first two, there is a high chance that we will have the third.

Ultimately, our extensive study of genres allowed us to remove eight of the original labels, reducing

the number of genres from 35 to 27.

1.4 2.2 - Data Split

In this subpart, we discuss data splitting strategies in the context of our multilabel situation. In our case, a classic 60-20-20 split or a stratified split is not necessarily the best option, since multiple labels can appear, making a balanced division difficult.

The initial idea was to create a training, development and test set for each possible label. However, this would combine the training and test data from one classifier to another, and it is impossible to completely avoid interdependence in our context.

This is why we will use `scikit multilearn` `iterative_train_test_split` function as suggested on the [Data Science Stack Exchange](https://github.com/scikit-multilearn/scikit-multilearn/discussions/282), which provide an optimised version of the classical train test split for multilabel. Note that, in opposition with standard train-test split method, the `iterative` method, described by *Konstantinos Sechidis et al.* in the paper [On the Stratification of Multi-label Data](#), is usually deterministic and aims to conserve the balance of the different labels in the produced sets.

Unfortunately, in the event of a tie, the break is chosen at random. From version 0.2.0 of `scikit-multilearn`, the implemented function does not enable us to fix the state. As a result, we will temporarily fix the state globally using `np.random.seed` (after checking, this seems to work). For simplicity and due to time constraints, we will evaluate our model using a tie-breaking seed of 0 for the training/dev-test split and a seed of 42 for the dev/test split. Considering the nature of the split method, which is mostly deterministic, this should not have significant consequences.

In our case, with a large number of classes and a reasonably sized dataset, we will use a 75-15-10 repartition.

```
[17]: # To use the function: https://github.com/scikit-multilearn/scikit-multilearn/
      ↪ discussions/282

      # Transform our labels lists into dummy classifier
      mlb = MultiLabelBinarizer()
      Y = mlb.fit_transform(df_genresFiltered.loc[:, "genres"].tolist())

      # Transform our X into something working
      X = df_genresFiltered[["description"]].to_numpy()
      X = X[... , np.newaxis]

      # Register the id of each labels (column of y)
      labelIdtoLabelName = {i:label for i, label in enumerate(mlb.classes_)}
      labelNameToLabelId = {v:k for k,v in labelIdtoLabelName.items()}

      # Train Dev Test Split
      ## Train Test Split (0.75 training)
      ## Ensure reproducibility globally
      np.random.seed(0)
```

```

X_train, y_train, X_test, y_test = iterative_train_test_split(X, Y, test_size =
    ↪0.25)
## Test Dev split (0.6*0.25=0.15 dev and 0.4*0.25 = 0.1 test)
np.random.seed(42)
X_test, y_test, X_dev, y_dev = iterative_train_test_split(X_test, y_test,
    ↪test_size=0.6)

# Reset the seed
np.random.seed(None)

# Print some informations:
print(
    f"""
    The shape of the training set is {X_train.shape},
    the shape of the test set is {X_test.shape},
    the shape of the dev set is {X_dev.shape}
    """
)

```

The shape of the training set is (8765, 1, 1),
 the shape of the test set is (1210, 1, 1),
 the shape of the dev set is (1795, 1, 1)

1.5 2.3 - Preprocessing Strategies

In this section, we will present our preprocessing choices in a bullet-point format with justification.

- **Tokenization Method:** Word-level tokenisation. We do not plan to use transformers in our multilabel context as this approach would be too time-consuming without a GPU. Therefore, we will stick to classical machine learning (ML) techniques such as TF-IDF, Bayes and logistic regressions, as well as pre-trained embeddings like GloVe, which is not trained on subwords. Additionally, in Section 2.1 bis, we observed many composed words with rare occurrences. Consequently, we will remove the hyphen between composed words to break them down into separate tokens. Also, we will lowercase the words to standardise different versions of the same word.
- **Lemmatization vs Stemming:** In our case, where we hypothesise that the context of a label is present in the semantics of the tokens, and where we can handle higher time complexity because the texts are not too long, we have chosen to use lemmatisation. Indeed, it could yield better results than stemming because it is context-aware, unlike stemming.
- **Stopwords strategy:** In Section 2.1bis, we observed that classical stop words (such as *A*, *The*, *To*, *And*, etc.) are very frequent without conveying a particular meaning. In this case, we will use the nltk English stopword set to filter them out.
- **Rare word strategy:** In Section 2.1bis, we discovered that more than half of the words are very rare and/or specialised. In this context, we could use TF-IDF to identify and filter out the less important words based on a proportion or threshold.
- **Handling of emoji and punctuation:** Since we are not conducting a sentiment analysis, emojis (which are almost non-existent in our dataset) and punctuation do not reveal particularly strong semantics. We will therefore remove them. The same will be done for numbers in general.

With that in mind, we can now preprocess our dataset.

```
[18]: def preprocess(string:str, morphologicalAnalysis:bool = False) -> np.
      ndarray[str]:
          # Preprocess the string using SpaCy and en_cor_web_sm
          string = nlp(str(string))

          # If morphological analysis, stop here
          if morphologicalAnalysis:
              return string

          # Get the lemma from the
          lemmatisedString = " ".join([token.lemma_ for token in string])

          # Match all words that are not digits. Match accentuated characters and
          words with apostrophes (don't, won't ,...)
          strings = re.findall(r"^[^W\d_]+(?:'[^W\d_]+)*", lemmatisedString)

          # Return and remove stop words
          return [word.lower() for word in strings if word.lower() not in stop_words]

# Just so that we know the time spent for preprocessing
begin = time.time()

# Preprocess the test set
preprocessedX_test = [preprocess(X_test[i,0,0]) for i in range(X_test.shape[0])]
print(f"Preprocessing of Test Set done! [{X_test.shape[0]} in {time.
      time()-begin:.4f}s]")
begin = time.time()

# Preprocess the dev set
preprocessedX_dev = [preprocess(X_dev[i,0,0]) for i in range(X_dev.shape[0])]
print(f"Preprocessing of Dev Set done! [{X_dev.shape[0]} in {time.time()-begin:.
      4f}s]")
begin = time.time()

# Preprocess the training set
preprocessedX_train = [preprocess(X_train[i,0,0]) for i in range(X_train.
      shape[0])]
print(f"Preprocessing of Train Set done! [{X_train.shape[0]} in {time.
      time()-begin:.4f}s]")
```

Preprocessing of Test Set done! [1210 in 21.6501s]

Preprocessing of Dev Set done! [1795 in 31.2069s]

Preprocessing of Train Set done! [8765 in 151.7110s]

Now that we have successfully preprocessed the text, the next step is to tokenise it. To do this, we will first create a vocabulary based on the training set.


```
[19]: # Counter of word from training set
trainingWordCounter = Counter(chain.from_iterable(preprocessedX_train))

# Get a vectorizer
vectorizer = TfidfVectorizer(
    tokenizer = lambda x: x, # Input as tokens
    preprocessor = lambda x: x, # Skip preprocessing (already done)
    token_pattern = None # Disable tokenizer (we already have tokens)
)

# Get Tfidf
preprocessingTFIDFMatrix = vectorizer.fit_transform(preprocessedX_train)

# Get the vocabulary mapping
trainingVocab = vectorizer.vocabulary_
inversedTrainingVocab = {v:k for k,v in trainingVocab.items()}

# Get the initial size of the training vocabulary
print(f"Size of the vocabulary: {len(trainingVocab)}")
```

Size of the vocabulary: 26449

From now on, we have the base vocabulary. However, it is still too large. Even after removing stop words, lemmatising and selecting only the training set vocabulary, we went from 41,025 tokens in the whole dataset to 26,449.

To solve this issue, we will set a threshold for the average TF-IDF per token. Indeed, less important tokens tend to have lower tf-idf values.

```
[20]: # The minimal threshold on tf-idf for which a token is not considered rare
    ↪ anymore.
# This threshold is purely arbitrary and could be used as an hyperparameter if
    ↪ the time/Processing time is on our side
# This threshold was chose to observe token that are not atrociously rare while
    ↪ being important in the semantics. By this
# threshold, we have approximately 2.2k vocabulary size, which is also
    ↪ reasonable for our system.
threshold = 0.0005 #0.0005

# Get the vocab above the threshold in term of average tf-idf
filteredVocabID = np.argwhere(preprocessingTFIDFMatrix.mean(axis=0)>threshold)[:
    ↪ ,1]
filterVocab = set([inversedTrainingVocab[i] for i in filteredVocabID])

# Plot the results of the selection
pd.DataFrame(
    list(sorted([
        vocab,
```

```

        trainingWordCounter[vocab],
        preprocessingTFIDFMatrix[:, trainingVocab[vocab]].mean(axis=0).round(6).
→tolist()[0][0]
    ) for vocab in filterVocab], key=lambda x: x[2], reverse=True)),
    columns=["Token", "Frequency", "Average tf-idf"]
)

```

```

[20]:
      Token  Frequency  Average tf-idf
0      world      5370      0.026643
1       life      4474      0.021673
2        one      4411      0.019427
3       love      3238      0.018241
4    become      3513      0.016915
...
2184 descendant      40      0.000503
2185  activity      39      0.000502
2186 enormous      37      0.000502
2187 virtuous      35      0.000501
2188      soar      36      0.000501

```

[2189 rows x 3 columns]

Now that we have selected the worthy tokens, we can filter them depending on whether they are outside the vocabulary, i.e. marked with the code <oo>.

```

[21]: # Filter <oo> for training set
preprocessedFilteredX_train = [
    list(map(lambda x: "<oo>" if x not in filterVocab else x, doc)) for doc in
→preprocessedX_train
]

# Filter <oo> for dev set
preprocessedFilteredX_dev = [
    list(map(lambda x: "<oo>" if x not in filterVocab else x, doc)) for doc in
→preprocessedX_dev
]

# Filter <oo> for test set
preprocessedFilteredX_test = [
    list(map(lambda x: "<oo>" if x not in filterVocab else x, doc)) for doc in
→preprocessedX_test
]

# Check if it worked
print(preprocessedFilteredX_test[0])

```

```

['luo', 'ning', 'cannon', 'fodder', 'female', 'support', 'character',
'entertainment', 'circle', 'love', 'novel', 'reborn', 'refuse', 'cannon',

```

```
'fodder', '<oov>', 'fan', '<oov>', 'luo', 'ning', 'flower', '<oov>', '<oov>',
'dance', 'act', 'every', 'day', 'know', 'tie', 'top', 'celebrity', 'gain',
'<oov>', 'nothing', 'good', 'look', 'later', 'luo', 'ning', 'appear', 'front',
'public', '<oov>', 'dance', '<oov>', '<oov>', 'four', 'art', 'survive', 'wild',
'cook', 'delicious', 'food', 'draw', '<oov>', 'look', 'beautiful', 'high', 'iq',
'super', '<oov>', 'acting', 'skill', '<oov>', 'fan', 'get', 'good', 'slap',
'face', 'big', 'sister', 'choose', 'suddenly', 'one', 'day', '<oov>', 'search',
'go', '<oov>', '<oov>', 'mysterious', 'king', 'silver', 'screen', 'lu', 'xun',
'kiss', 'luo', 'ning', '<oov>', 'release', '<oov>', 'quickly', 'become', 'top',
'<oov>', '<oov>', 'non', 'fan', '<oov>', 'could', 'believe', 'could', '<oov>',
'must', '<oov>', 'luo', 'ning', 'try', 'tie', 'good', 'actor', 'gain', '<oov>',
'<oov>', '<oov>', '<oov>', 'limit', '<oov>', 'luo', 'ning', 'could', '<oov>',
'lu', 'xun', '<oov>', 'post', '<oov>', '<oov>', 'first', '<oov>', '<oov>',
'real', 'shameless', 'person', 'everyone', 'could', 'see', 'good', 'actor',
'show', 'relationship', '<oov>', 'different', 'way', 'every', 'day', 'whether',
'show', 'luo', 'ning', 'delicious', 'food', 'beautiful', '<oov>', 'fan',
'public', 'strike', 'lightning', 'expect', 'kind', 'good', 'actor']
```

This sanity check clearly shows that it works. If we want to proceed to the next steps, we will need to vectorise these texts. For this, we will use the tf-idf method again.

```
[22]: # Vectorise training set
X_train_tfidfVectorised = vectorizer.fit_transform(preprocessedFilteredX_train)

# Sanity check. Check whether the size of the vocabulary is the same (if we add
↳oov)
N = vectorizer.vocabulary_.keys().__len__()
print(X_train_tfidfVectorised.shape, X_train_tfidfVectorised.shape[1]==N)

# Vectorise dev set and check
X_dev_tfidfVectorised = vectorizer.transform(preprocessedFilteredX_dev)
print(X_dev_tfidfVectorised.shape, X_dev_tfidfVectorised.shape[1]==N)

# Vectorise test and check
X_test_tfidfVectorised = vectorizer.transform(preprocessedFilteredX_test)
print(X_test_tfidfVectorised.shape, X_test_tfidfVectorised.shape[1]==N)
```

```
(8765, 2190) True
```

```
(1795, 2190) True
```

```
(1210, 2190) True
```

Vocabulary Size: Ultimately, the vocabulary size decreased from 41,025 to 2,190 words (including <oov>), representing a reduction of around 95%. The proportion of <oov> can be easily calculated as shown below.

```
[23]: # Count occurrences in each dataset
trainCount = Counter(chain.from_iterable(preprocessedFilteredX_train))
devCount = Counter(chain.from_iterable(preprocessedFilteredX_dev))
testCount = Counter(chain.from_iterable(preprocessedFilteredX_test))
```

```
# Retrieve OOV proportion.
print(f"Training Set: #<oov>={trainCount['<oov>']}, #token={trainCount.
    ↳total()}, <oov> proportion: {trainCount['<oov>']/trainCount.total()*100:.
    ↳4f}%")
print(f"Dev Set: #<oov>={devCount['<oov>']}, #token={devCount.total()}, <oov>
    ↳proportion: {devCount['<oov>']/devCount.total()*100:.4f}%")
print(f"Test Set: #<oov>={testCount['<oov>']}, #token={testCount.total()},
    ↳<oov> proportion: {testCount['<oov>']/testCount.total()*100:.4f}%")
```

```
Training Set: #<oov>=110623, #token=574402, <oov> proportion: 19.2588%
Dev Set: #<oov>=22724, #token=116582, <oov> proportion: 19.4919%
Test Set: #<oov>=15074, #token=77449, <oov> proportion: 19.4631%
```

Here, the <oov> proportion is consistent among the three datasets and reaches about 20%. This seems enormous, until we remember that we intentionally classified rare words into <oov> to prune the vocabulary. Just to avoid recomputing all of this, we will now store these sets.

```
[24]: # Save the X sets with tf-idf
with open("pickledFiles/X_train_tfidfVectorised.pkl", "wb") as file:
    pickle.dump(X_train_tfidfVectorised, file)
with open("pickledFiles/X_dev_tfidfVectorised.pkl", "wb") as file:
    pickle.dump(X_dev_tfidfVectorised, file)
with open("pickledFiles/X_test_tfidfVectorised.pkl", "wb") as file:
    pickle.dump(X_test_tfidfVectorised, file)

# Save the y sets
with open("pickledFiles/y_train.pkl", "wb") as file:
    pickle.dump(y_train, file)
with open("pickledFiles/y_dev.pkl", "wb") as file:
    pickle.dump(y_dev, file)
with open("pickledFiles/y_test.pkl", "wb") as file:
    pickle.dump(y_test, file)

# Save the labels
with open("pickledFiles/labelNameToLabelId_labelIdToLabelName.pkl", "wb") as
    ↳file:
    pickle.dump((labelNameToLabelId, labelIdToLabelName), file)
with open("pickledFiles/classes_.pkl", "wb") as file:
    pickle.dump(mlb.classes_, file)
```

From this observation, this seems reasonable. For our second modelling, we will use the [GloVe](#) embedding 2024 Wikipedia + Gigaword 5 (11.9B tokens, 1.2M vocab, uncased, 100d vectors, 560 MB download) available [here](#) and published in the paper *A New Pair of GloVes* by *Riley Carlson et al.*

GloVe's particular feature is that its embedding can understand the underlying linear structures between words. For example, 'King-Man' plus 'Woman' would be approximately equal to 'Queen'. This structure will be useful for providing a document embedding, which we will use for classifica-

tion.

Note (How to download GloVe): when downloaded, unzip it, and place the obtained file in the folder GloVe after renaming it into GloVe2024.txt

```
[25]: # Load GloVe
# Solution adapted from https://stackoverflow.com/questions/37793118/
# load-pretrained-glove-vectors-in-python
class GloVeClass():
    def __init__(self, path: str):
        self.GloVe = pd.read_table(path, sep=" ", index_col=0, header=None,
        quoting=csv.QUOTE_NONE)
        self.GloVeMatrix = self.GloVe.to_numpy()
        self.dim = self.GloVe.shape[1]

        # Precompute normalized matrix to make closest word faster
        norms = np.linalg.norm(self.GloVeMatrix, axis=1, keepdims=True)
        self.GloVeMatrixNorm = self.GloVeMatrix / norms

    def closestWord(self, vec: np.ndarray) -> str:
        # Normalise the input vector
        vec_norm = vec / np.linalg.norm(vec)

        # Compute cosine similarity as only dot product since it is already
        # normalised (||A||*||B||)
        cos = np.dot(self.GloVeMatrixNorm, vec_norm)

        # Get the index with the highest cosine similarity
        i = np.argmax(cos)

        # Get the word
        return self.GloVe.iloc[i].name

    def getVector(self, tokens: list[str]) -> np.ndarray:
        # If the list is empty, return nan for the document embedding
        if len(tokens) == 0:
            return np.full((1, self.dim), np.nan)

        # By defaults, embedding is unknown
        result = np.full((len(tokens), self.dim), np.nan)

        # Find which tokens exist
        mask = [t in self.GloVe.index for t in tokens]

        # If tokens exists
        if any(mask):
            # Get the terms
            found_tokens = [t for t, m in zip(tokens, mask) if m]
```

```

        # Place the vector at the right position
        result[mask] = self.GloVe.loc[found_tokens].to_numpy()

    return result

    def getCosineSimilarity(self, vec1:np.ndarray, vec2:np.ndarray) -> float:
        return np.dot(vec1, vec2)/(np.linalg.norm(vec1)*np.linalg.norm(vec2))

# Check whether GloVe was well imported
GloVe = GloVeClass(path="GloVe/GloVe2024.txt")

```

Now that we have imported GloVe, we will just carry out a few basic checks. - If I take a Pizza, remove the Italian component and replace them with the Japanese one, will it become Sushi? - What happen if I give an empty list of string? - What happen if an unknown word is given?

```

[26]: # We indeed get sushis
print(GloVe.closestWord((GloVe.getVector(["pizza"])-GloVe.
    ↪getVector(["italy"])+GloVe.getVector(["japan"]))[0]))

# We get a full nan vector
print(np.nanmean(GloVe.getVector([]), axis=0))

# The mean of nan give nan
print(np.nanmean(GloVe.getVector(["<oov>", "<oov>"]), axis=0))

# The mean of nan and known words give the mean without nan.
print(np.nanmean(GloVe.getVector(["pizza", "<oov>"]), axis=0))

```

sushi

```

[nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan]
[nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan
nan nan nan nan nan nan nan nan nan nan]
[-0.597682  0.800201 -0.821124 -0.629171 -0.348775 -0.183689  0.491192
-1.639139 -0.350868  0.745271 -0.168258  0.451539 -0.46893  0.666111
 0.06937  -0.207809  0.600642  0.253673  0.671275  0.007801 -0.593017
-1.295505 -0.338431  0.415013  0.097705  0.425839  0.445962  0.012131
 0.976418  0.593613 -0.247837 -0.121297  0.142813  0.720986 -1.485135
 0.054065 -0.592971 -0.473699 -0.301581  0.609923  0.600408 -0.149095]

```

```
-0.221138  0.256235 -0.643963 -0.301492 -0.851119  0.038139  1.050478
-0.119473  0.234288 -0.018173 -0.263931  0.26988  -0.486657  0.2017
-1.100795  0.497336  0.981733 -0.015843  0.441305 -0.572203  0.77742
-0.577814 -0.589868 -0.606523 -0.913109 -0.030291  0.913567  0.025959
 0.635317 -0.005309  0.04555  -0.545749  0.047615  0.444095  0.674614
-0.146463 -0.934049  0.058844  1.061886  0.439818 -0.848131  0.780473
-0.042284 -0.442385 -0.241283 -1.145398  0.02799  0.267519 -0.177688
 0.204612 -0.538658  0.640832 -0.106214 -0.213173 -0.706105  0.106571
 0.140512 -0.886143]
```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\248797406.py:5: RuntimeWarning:
Mean of empty slice

```
print(np.nanmean(GloVe.getVector([]), axis=0))
```

C:\Users\arnol\AppData\Local\Temp\ipykernel_7700\248797406.py:8: RuntimeWarning:
Mean of empty slice

```
print(np.nanmean(GloVe.getVector(["<oov>", "<oov>"]), axis=0))
```

Yes, we get sushi. If we receive an empty list, it simply returns a vector of NaN; the same applies to an unknown term.

Now, back to something serious. Since the underlying structure can be considered captured, we would like to embed all our documents using this method instead of TF-IDF. Still, we will keep the word filtering.

WARNING: This section is very long to run (>4h). Use the precomputed pickle instead.

```
[27]: # Get Document embedding for training
print("---Training---")
begin = time.time()
X_train_GloVeVectorised = []

# For each document
for i, trainDocument in enumerate(preprocessedFilteredX_train):
    # Inform the user
    if i%100 == 0:
        print(f"{i}/{len(preprocessedFilteredX_train)} [{time.time()-begin:.5f}s]")

    # Get the average document vector ignoring <oov>.
    X_train_GloVeVectorised.append(np.nanmean(GloVe.getVector(trainDocument),
↪axis=0))

# Normalise the resulting vectors
X_train_GloVeVectorised = X_train_GloVeVectorised/np.linalg.
↪norm(X_train_GloVeVectorised, axis=1, keepdims=True)

# Save the results
with open("pickledFiles/X_train_GloVeVectorised.pkl", "wb") as file:
```

```
pk1.dump(X_train_GloVeVectorised, file)
```

```
---Training---
```

```
0/8765 [0.00000s]
```

```
100/8765 [89.73562s]
```

```
200/8765 [177.77572s]
```

```
300/8765 [271.03638s]
```

```
400/8765 [381.38519s]
```

```
500/8765 [474.96868s]
```

```
600/8765 [565.29262s]
```

```
C:\Users\arnol\AppData\Local\Temp\ipykernel_3808\1765558322.py:13:
```

```
RuntimeWarning: Mean of empty slice
```

```
  X_train_GloVeVectorised.append(np.nanmean(GloVe.getVector(trainDocument),  
axis=0))
```

```
700/8765 [666.96530s]
```

```
800/8765 [758.05883s]
```

```
900/8765 [850.89824s]
```

```
1000/8765 [954.49263s]
```

```
1100/8765 [1054.78085s]
```

```
C:\Users\arnol\AppData\Local\Temp\ipykernel_3808\1765558322.py:13:
```

```
RuntimeWarning: Mean of empty slice
```

```
  X_train_GloVeVectorised.append(np.nanmean(GloVe.getVector(trainDocument),  
axis=0))
```

```
1200/8765 [1141.90673s]
```

```
C:\Users\arnol\AppData\Local\Temp\ipykernel_3808\1765558322.py:13:
```

```
RuntimeWarning: Mean of empty slice
```

```
  X_train_GloVeVectorised.append(np.nanmean(GloVe.getVector(trainDocument),  
axis=0))
```

```
C:\Users\arnol\AppData\Local\Temp\ipykernel_3808\1765558322.py:13:
```

```
RuntimeWarning: Mean of empty slice
```

```
  X_train_GloVeVectorised.append(np.nanmean(GloVe.getVector(trainDocument),  
axis=0))
```

```
-----  
KeyboardInterrupt
```

```
Traceback (most recent call last)
```

```
Cell In[27], line 13
```

```
    10         print(f"{i}/{len(preprocessedFilteredX_train)} [{time.  
↳time()-begin:.5f}s]")  
    12         # Get the average document vector ignoring <oov>.  
--> 13         X_train_GloVeVectorised.append(np.  
↳nanmean(GloVe.getVector(trainDocument), axis=0))  
    15 # Normalise the resulting vectors  
    16 X_train_GloVeVectorised = X_train_GloVeVectorised/np.linalg.  
↳norm(X_train_GloVeVectorised, axis=1, keepdims=True)
```



```

Cell In[25], line 35, in GloVeClass.getVector(self, tokens)
    32 result = np.full((len(tokens), self.dim), np.nan)
    34 # Find which tokens exist
--> 35 mask = [t in self.GloVe.index for t in tokens]
    37 # If tokens exists
    38 if any(mask):
    39     # Get the terms

```

KeyboardInterrupt:

```

[29]: # Get Document embedding for dev
print("---Dev---")
begin = time.time()
X_dev_GloVeVectorised = []

# For each document
for i, devDocument in enumerate(preprocessedFilteredX_dev):
    # Inform the user
    if i%100 == 0:
        print(f"{i}/{len(preprocessedFilteredX_dev)} [{time.time()-begin:.
↪5f}s]")

        # Get the average document vector ignoring <ooV>.
        X_dev_GloVeVectorised.append(np.nanmean(GloVe.getVector(devDocument),
↪axis=0))

# Normalise the resulting vectors
X_dev_GloVeVectorised = X_dev_GloVeVectorised/np.linalg.
↪norm(X_dev_GloVeVectorised, axis=1, keepdims=True)

# Save the results
with open("pickledFiles/X_dev_GloVeVectorised.pkl", "wb") as file:
    pickle.dump(X_dev_GloVeVectorised, file)

```

```

---Dev---
0/1795 [0.00000s]
100/1795 [116.00680s]
200/1795 [214.00428s]
300/1795 [306.88523s]
400/1795 [412.24012s]
500/1795 [515.76766s]
600/1795 [616.98793s]
700/1795 [722.78656s]
800/1795 [836.39690s]
900/1795 [935.72032s]
1000/1795 [1044.14062s]
1100/1795 [1146.71399s]

```

```

1200/1795 [1257.79577s]
1300/1795 [1360.14620s]
1400/1795 [1475.26936s]
1500/1795 [1583.21149s]
1600/1795 [1693.87401s]
1700/1795 [1785.03065s]

```

```

[30]: # Get Document embedding for dev
print("---Test---")
begin = time.time()
X_test_GloVeVectorised = []

# For each document
for i, testDocument in enumerate(preprocessedFilteredX_test):
    # Inform the user
    if i%100 == 0:
        print(f"{i}/{len(preprocessedFilteredX_test)} [{time.time()-begin:.
↪5f}s]")

        # Get the average document vector ignoring <ooV>.
        X_test_GloVeVectorised.append(np.nanmean(GloVe.getVector(testDocument),
↪axis=0))

# Normalise the resulting vectors
X_test_GloVeVectorised = X_test_GloVeVectorised/np.linalg.
↪norm(X_test_GloVeVectorised, axis=1, keepdims=True)

# Save the results
with open("pickledFiles/X_test_GloVeVectorised.pkl", "wb") as file:
    pickle.dump(X_test_GloVeVectorised, file)

```

```

---Test---
0/1210 [0.00000s]
100/1210 [104.78642s]
200/1210 [200.12514s]
300/1210 [315.99783s]
400/1210 [418.60039s]
500/1210 [519.60708s]
600/1210 [620.58360s]
700/1210 [720.06758s]
800/1210 [826.18177s]
900/1210 [928.20169s]
1000/1210 [1024.20933s]
1100/1210 [1117.44494s]
1200/1210 [1219.29888s]

```

1.5.1 Pickle restoration point

This is the restoration point to avoid recomputing all of this

```
[27]: # Load the classes of the multilabel binarizer
with open("pickledFiles/classes_.pkl", "rb") as file:
    classes_:np.ndarray[str] = pickle.load(file)

# Get the label to id dicts
with open("pickledFiles/labelNameToLabelId_labelIdToLabelName.pkl", "rb") as file:
    fileContent = pickle.load(file)
    labelNameToLabelId:dict[str,int] = fileContent[0]
    labelIdToLabelName:dict[int,str] = fileContent[1]

## Get the tf-idf sets
with open("pickledFiles/X_train_tfidfVectorised.pkl", "rb") as file:
    X_train_tfidfVectorised:np.ndarray[float] = pickle.load(file)
with open("pickledFiles/X_dev_tfidfVectorised.pkl", "rb") as file:
    X_dev_tfidfVectorised:np.ndarray[float] = pickle.load(file)
with open("pickledFiles/X_test_tfidfVectorised.pkl", "rb") as file:
    X_test_tfidfVectorised:np.ndarray[float] = pickle.load(file)

## Get the GloVe sets
with open("pickledFiles/X_train_GloVeVectorised.pkl", "rb") as file:
    X_train_GloVeVectorised:np.ndarray[float] = pickle.load(file)
with open("pickledFiles/X_dev_GloVeVectorised.pkl", "rb") as file:
    X_dev_GloVeVectorised:np.ndarray[float] = pickle.load(file)
with open("pickledFiles/X_test_GloVeVectorised.pkl", "rb") as file:
    X_test_GloVeVectorised:np.ndarray[float] = pickle.load(file)

## Get y sets
with open("pickledFiles/y_train.pkl", "rb") as file:
    y_train:np.ndarray[int] = pickle.load(file)
with open("pickledFiles/y_dev.pkl", "rb") as file:
    y_dev:np.ndarray[int] = pickle.load(file)
with open("pickledFiles/y_test.pkl", "rb") as file:
    y_test:np.ndarray[int] = pickle.load(file)
```

Now, let's check whether the GloVe embedding worked. To check this, we will take a look at one instance and its shape. We will also test the number of NaNs that we will have to drop in our datasets. (NaN embedding just means that either the embedding is full of <ooV>, no embedding has been found, or no description has been found.)

```
[28]: print(X_test_GloVeVectorised.shape)
print(X_test_GloVeVectorised[1,:])
print(f"Number of missing entries (Training): {np.
    isnan(X_train_GloVeVectorised).sum()/GloVe.dim}")
```

```
print(f"Number of missing entries (Dev): {np.isnan(X_dev_GloVeVectorised).sum()/
↳GloVe.dim}")
print(f"Number of missing entries (Test): {np.isnan(X_test_GloVeVectorised).
↳sum()/GloVe.dim}")
```

```
(1210, 100)
```

```
[ 0.01359042  0.02806792 -0.04619222  0.07259577  0.029259    0.0010169
 0.08722882 -0.0429012   0.07853468  0.07057988 -0.02524143  0.01909653
-0.02352157 -0.02707391 -0.1282359   -0.01826146 -0.00077738  0.09680885
 0.03705659  0.0228977   -0.21966097 -0.01407445 -0.02342482  0.0204378
-0.08574691  0.16786683 -0.05745785 -0.08459545  0.00059815 -0.05860269
-0.03509958  0.03293342  0.1330228   -0.05577178 -0.5627161   -0.06396027
 0.09425551 -0.01386239 -0.04886821 -0.00115146 -0.13405699 -0.10983001
 0.08528798 -0.07696635  0.00617978  0.10077662 -0.01033531  0.02328978
 0.41425138 -0.00079269 -0.0569964   -0.11722021 -0.129288    0.15421793
 0.04008615 -0.12725404  0.04174658 -0.01090204 -0.03550508  0.04877824
 0.11877907 -0.03051125 -0.13815396  0.04302495 -0.08192396  0.13269686
-0.0052358   0.00874607  0.0919222   0.05606102  0.02338914 -0.02785002
-0.00497118  0.05402683  0.00946504 -0.03945296 -0.07432896 -0.00452235
-0.06905409  0.00243031 -0.00455187  0.09240954  0.03491285  0.13643513
-0.01604874  0.07987674  0.04515377  0.03606937 -0.0814846   -0.04729428
-0.10413189  0.03070192  0.09358191  0.01534658 -0.08008904  0.02739227
-0.0998901   0.05670913  0.01696265 -0.0479105 ]
```

```
Number of missing entries (Training): 8.0
```

```
Number of missing entries (Dev): 0.0
```

```
Number of missing entries (Test): 0.0
```

Since the presence of unknown embeddings is quite low, we will simply ignore them when designing the classification models.

Regarding the vocabulary size, although we initially have the same vocabulary size as with TF-IDF, ultimately, we do not reason in terms of vocabulary anymore, but rather in terms of vectors of fixed length (100). This can be useful as it compresses our data in a way.

The pros and cons of this approach are that document embeddings allow us to work with smaller vectors and concentrate information. The potential drawback is clear: the longer the document, the more diluted the information. Fortunately, given the nature of the descriptions (short, focused texts), we do not expect to face this problem.

We will now answer the **thinking questions** of this part: 1. **Which preprocessing decision most affects performance? Why?:** In our case, the preprocessing decisions that most affect performance are clearly the rare word filtering. It is never easy to set a clear threshold below which a word should not be taken into account. If the threshold is too high, we might not be able to distinguish between our data due to a lack of vocabulary. On the other hand, choosing a low threshold leads to a vocabulary explosion, which drastically reduces performance in terms of time and causes us to lose focus on our model. 2. **When does aggressive cleaning hurt performance?:** If we reduce the vocabulary size too much, it becomes difficult to classify documents, especially given our high proportion of rare words. In our case, increasing the proportion of <ooV> could significantly impact performance. 3. **Does removing stopwords always help? Provide evidence.:** In our case, it helps because the description is full of specialised words. Unfortunately, this is not always

the case. For example, “This is not important” and “This is important” are clearly opposites, but using **nltk** stopwords, both would be categorised as “important”. Clearly, the negation semantics are lost here.

```
[29]: # Suppression of the training with nan in both y and X
toRemove = np.unique(np.where(np.isnan(X_train_GloVeVectorised))[0])
X_train_GloVeVectorisedOutNA = np.delete(X_train_GloVeVectorised, toRemove,
↪axis=0)
y_trainGloVeVectorisedOutNA = np.delete(y_train, toRemove, axis=0)
```

1.6 3 - Exploratory & Linguistic Analysis

For the analysis required in this section, please refer to **Section 2.1bis**, where you will find an interpretation of the frequency distributions by class, dependency structure statistics and word distribution analysis.

Here, we will provide a brief overview of our findings, but please refer to **Section 2.1bis** for a more detailed analysis.

- **Frequency distributions by class:** First, as a reminder, we want to categorise our description using the correct genre label from a set of 35 options. From a bar plot of genre frequencies, we observed highly heterogeneous data with tags that were present in more than half of the dataset, such as **Romance** or **Fantasy**. At the other end of the spectrum, after a Poisson-distributed decrease in occurrences, we find very rare tags such as **Wuxia**, **Shojo Ai** and **Sports**, which account for much less than 1% of occurrences. After unravelling this fact, we merged and dropped some labels (to reduce time strain on our models and increase the possibility of distinguishing between classes). These drops concern only low-occurrence genres, while merges happen with closely related genres. For example, if you have a sofa in your house, you could also say that you have a couch, but you wouldn’t say that you have both. Similarly, some genres, such as **Shojo Ai** and **Yuri**, essentially refer to the same problem.
- **Dependency structure statistics:** In a multilabel context, it is important to understand how different labels relate to each other. Indeed, it could be useful to be aware of such relationships when designing a post-processing step that takes correlation into account to enrich our results. In the previously mentioned section, after dropping or merging genres, we studied the correlation between classes, and more precisely, how often does a label appear alongside another label? In our analysis, we found that typical domain structures such as **Fantasy**, **Action** and **Adventure** were closely tied together, as were **Fantasy**, **Comedy** and **Romance**. This is interesting because it suggests that the semantics of these labels are linked. Additionally, we observed the opposite: genres that do not connect to each other. This is typically the case with **Sci-fi**, **Tragedy** and **Horror**, for example, which are almost not linked together in any way.
- **Word Distribution Analysis:** Of the 1,499,431 words in the entire dataset, 41,025 were unique and 733,681 were stopwords (almost half). This observation made the use of stopwords imperative in our context in order to retrieve the real semantics. Additionally, we observed that the number of occurrences of the 41,025 unique words was also heterogeneous, ranging from highly occurring words related to certain genres, such as *World*, to the least occurring words due to their differing natures (compound words, specialised terms or non-English words). In terms of word frequency, we found that only 12.3% of words appear more

than 20 times in the entire dataset, while more than half of unique words occur only once or twice. Ultimately, this information is important for setting a threshold on vocabulary size.

Mandatory Linguistic Component For the mandatory component analysis, we will observe whether imperative is used more frequently for certain classes. This was also done in the following post on [Stack Overflow](#), we will identify our imperative form as a verb being at the root of the sentence (as an approximation).

```
[30]: # Get the imperative estimates
X_imperative = []

# For each texts
for i, value in enumerate(X[:,0]):
    # Inform the user
    if i%1000==0:
        print(f"[{i}/{X.shape[0]}]")

    # Get the analysis from Spacy
    morpho = nlp(str(value[0]))

    # Get the imperative proportion
    X_imperative.append(sum([token.tag_ == "VB" and token.dep_=="ROOT" for
↪token in morpho])/len(morpho))

# Print head of result for sanity check
print(X_imperative[:5])
```

```
[0/11770]
[1000/11770]
[2000/11770]
[3000/11770]
[4000/11770]
[5000/11770]
[6000/11770]
[7000/11770]
[8000/11770]
[9000/11770]
[10000/11770]
[11000/11770]
[0.014044943820224719, 0.0, 0.0, 0.0, 0.010638297872340425]
```

Now that we have computed the proportions, we need to relate them to the different classes.

```
[31]: # Take the count and the proportion
count = np.zeros(Y.shape[1])
proportionSum = np.zeros(Y.shape[1])

# For each proportion
```

```

for i, value in enumerate(X_imperative):
    # Find the associated labels
    mask = np.argwhere(Y[i,:] == 0)

    # Update the values
    count[mask] += 1
    proportionSum[mask] += value

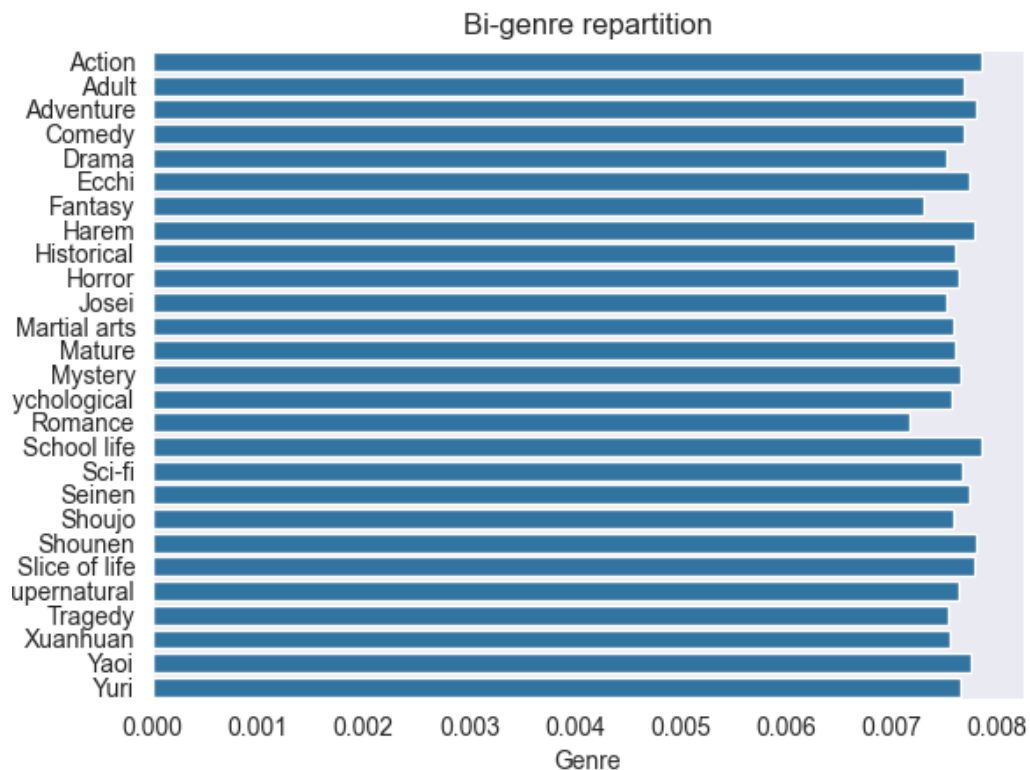
# Make a plot
plt.close("all")

sns.barplot(
    # Compute the average
    pd.DataFrame(pd.DataFrame([classes_, proportionSum/count]).T.to_numpy(),
        columns=["Label", "Imperative Proportion"],
        y="Label", x="Imperative Proportion"
    )

plt.title("Bi-genre repartition")
plt.xlabel("Genre")
plt.ylabel("Genre")

plt.show()

```



In this plot, we can see that no label emerges when we compare the estimated proportion of imperatives. This is somewhat deceptive, as we could have hypothesised that **Action** or **Martial Arts** would contain more imperatives (for training, for example); however, we ultimately found no correlation.

Consequently, we can conclude that imperative does not have a specific meaning in descriptions compared to other words. This does not contradict any of our initial hypotheses.

1.7 4 - Predictive Modelling

1.7.1 4.1 - Baselines

Most frequent label baseline (Multinomial Naïve Bayes) To make classifications in cases involving multilabel data, we will use scikit-multilearn to wrap scikit-model execution. In short, rather than training one classifier per column ourselves, we will let skmultilearn do it to optimise the process.

```
[32]: # Create the classifier
bayesClassifier = BinaryRelevance(
    MultinomialNB(), # Wrap MultinomialNB
    require_dense=[True, True] # Require dense to speed up our process (we have
    ↪ sparse matrices, especially for tf-idf)
)

# Fit the training data and predict on test set for tfidf
bayesClassifier.fit(X_train_tfidfVectorised, y_train)
y_predBayesTfidf = bayesClassifier.predict(X_test_tfidfVectorised)

# Fit the training data and predict on test set for GloVe
try:
    bayesClassifier.fit(X_train_GloVeVectorisedOutNA,
    ↪ y_trainGloVeVectorisedOutNA)
    y_predBayesGloVe = bayesClassifier.predict(X_test_GloVeVectorised)
except ValueError as e:
    print(f"Negative value in the embedding, cannot proceed with Multinomial
    ↪ Bayes [{e}]")
```

Negative value in the embedding, cannot proceed with Multinomial Bayes [Negative values in data passed to MultinomialNB (input X)]

As we can see here, the Bayes classifier worked with our TF-IDF encoding, which can be interpreted as a count in some way. Unfortunately, this is not the case for GloVe, which possesses negative values in document embeddings that cannot therefore be interpreted as a count. This causes the classifier to fail for this representation. We will therefore use a different baseline: the closeness to the average embedding (Nearest neighbour in a sense).


```

[33]: class ClosenessDocEmbeddingClassifier():
    def __init__(self):
        pass

    def fit(self, X_train:np.ndarray[float], Y_train:np.ndarray[int]) -> None:
        # Initialise the label embedding
        self.averageEmbedding = {}

        # For each label
        for label in range(Y_train.shape[1]):
            # Find which rows has this label
            inLabel = np.where(Y_train[:,label])[0]

            # Take the average of document possessing it
            self.averageEmbedding[label] = np.mean(X_train[inLabel,:], axis=0)

    def calibrateThreshold(self, X_dev:np.ndarray[float], Y_dev:np.
→ ndarray[int], precision:int=10**4, verbose:bool = True) -> None:
        # Cut the space [0,1] to precision slice (to test all the thresholds)
        thresholds = np.linspace(0,1,precision)

        # Register the best Hamming similarity
        bestHamming = np.full(len(self.averageEmbedding.keys()), -1,
→ dtype=float)
        self.thresholds = np.full(len(self.averageEmbedding.keys()), -1,
→ dtype=float)
        begin = time.time()

        # For each threshold
        for i, threshold in enumerate(thresholds):
            # Warn the user if necessary
            if verbose and i%1000 == 0:
                print(f"[{time.time()-begin:.5f}s] threshold: {i}/{precision}")

            # Create the predicted matrix as 0
            Y_pred = np.zeros_like(Y_dev, dtype=int)

            # For each label
            for label, labelEmbedding in self.averageEmbedding.items():
                # Compute the cosine similarity
                labelCos = GloVe.getCosineSimilarity(X_dev,labelEmbedding)

                # Fill the prediction
                Y_pred[np.where(labelCos>=threshold)[0], label] = 1

            # Take Hamming similarity for each label and store it if it is
→ the best

```

```

        if 1 - hamming_loss(Y_dev[:,label], Y_pred[:,label]) >
↪bestHamming[label]:
            self.thresholds[label] = threshold

    def predict(self, X_test:np.ndarray[float]) -> np.ndarray[int]:
        # Initialise the predicted matrix as zeros
        Y_pred = np.zeros((X_test.shape[0], len(self.averageEmbedding.keys()))),
↪dtype=int)

        # For each labels
        for label, labelEmbedding in self.averageEmbedding.items():

            # Compute cosine similarity
            labelCos = GloVe.getCosineSimilarity(X_test, labelEmbedding)

            # Make predictions
            Y_pred[np.where(labelCos>=self.thresholds[label])[0], label] = 1

        # Return the prediction
        return Y_pred

```

Now, we can make our GloVe predictions. Please note that hyperparameter tuning is performed using the Hamming similarity due to the imbalance in the labels and the greater importance assigned to the more frequent ones.

Warning: since this can be long and/or memory intensive, you can go to the next restoration point for the results.

```

[ ]: # Create the classifier
embeddingClassifier = ClosenessDocEmbeddingClassifier()

# Fit the classifier and tune hyperparameter
embeddingClassifier.fit(X_train_GloVeVectorisedOutNA,
↪y_trainGloVeVectorisedOutNA)
embeddingClassifier.calibrateThreshold(X_dev_GloVeVectorised, y_dev)

# Predict the results
y_predEmbeddingGloVe = embeddingClassifier.predict(X_test_GloVeVectorised)

```

```

[0.00000s] threshold: 0/10000
[16.96633s] threshold: 1000/10000
[33.62548s] threshold: 2000/10000
[49.91134s] threshold: 3000/10000
[66.19873s] threshold: 4000/10000
[83.19448s] threshold: 5000/10000
[99.46270s] threshold: 6000/10000
[115.73263s] threshold: 7000/10000

```

```
[132.56837s] threshold: 8000/10000
[148.91427s] threshold: 9000/10000
```

We may also do the same for tf-idf to compare both:

```
[ ]: # Create the classifier
embeddingClassifier = ClosenessDocEmbeddingClassifier()

# Fit the classifier and tune hyperparameter
embeddingClassifier.fit(X_train_tfidfVectorised.toarray(), y_train)
embeddingClassifier.calibrateThreshold(X_dev_tfidfVectorised.toarray(), y_dev)

# Predict the results
y_predEmbeddingTfidf = embeddingClassifier.predict(X_test_tfidfVectorised.
↪toarray())
```

```
[0.00000s] threshold: 0/10000
[60.34328s] threshold: 1000/10000
[119.49843s] threshold: 2000/10000
[176.11877s] threshold: 3000/10000
[232.78377s] threshold: 4000/10000
[289.73221s] threshold: 5000/10000
[347.09333s] threshold: 6000/10000
[403.73293s] threshold: 7000/10000
[460.47002s] threshold: 8000/10000
[517.38333s] threshold: 9000/10000
```

```
[ ]: # Save the results for Bayes TFIDF
with open("pickledFiles/y_predBayesTfidf.pkl", "wb") as file:
    pickle.dump(y_predBayesTfidf, file)

# Save the results for Embedding Similarity GloVe
with open("pickledFiles/y_predEmbeddingGloVe.pkl", "wb") as file:
    pickle.dump(y_predEmbeddingGloVe, file)

# Save the results for TFIDF Embedding Similarity
with open("pickledFiles/y_predEmbeddingTfidf.pkl", "wb") as file:
    pickle.dump(y_predEmbeddingTfidf, file)
```

1.7.2 Pickle restoration point

```
[34]: # Load tf-idf Bayes results
with open("pickledFiles/y_predBayesTfidf.pkl", "rb") as file:
    y_predBayesTfidf = pickle.load(file)

# Load GloVe Embedding similarity results
with open("pickledFiles/y_predEmbeddingGloVe.pkl", "rb") as file:
    y_predEmbeddingGloVe = pickle.load(file)
```

```
# Load tf-idf Embedding similarity results
with open("pickledFiles/y_predEmbeddingTfidf.pkl", "rb") as file:
    y_predEmbeddingTfidf = pickle.load(file)

print(y_predBayesTfidf, y_predEmbeddingGloVe, y_predEmbeddingTfidf)
```

```
(8, 0)      1
(12, 0)     1
(20, 0)     1
(22, 0)     1
(25, 0)     1
(30, 0)     1
(31, 0)     1
(34, 0)     1
(41, 0)     1
(46, 0)     1
(48, 0)     1
(53, 0)     1
(56, 0)     1
(64, 0)     1
(65, 0)     1
(76, 0)     1
(86, 0)     1
(87, 0)     1
(88, 0)     1
(92, 0)     1
(93, 0)     1
(98, 0)     1
(104, 0)    1
(108, 0)    1
(110, 0)    1
:          :
(486, 25)   1
(515, 25)   1
(525, 25)   1
(536, 25)   1
(543, 25)   1
(546, 25)   1
(550, 25)   1
(576, 25)   1
(599, 25)   1
(639, 25)   1
(645, 25)   1
(677, 25)   1
(689, 25)   1
(741, 25)   1
(767, 25)   1
```

```

(855, 25)      1
(915, 25)      1
(944, 25)      1
(995, 25)      1
(1010, 25)     1
(1065, 25)     1
(1085, 25)     1
(1115, 25)     1
(1123, 25)     1
(1151, 25)     1 [[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
...
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]] [[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
...
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]]

```

Logistic regression This seems good, we can continue onto the next baseline: Logistic Regression.

```

[35]: # Create the classifier
logisticClassifier = BinaryRelevance(
    LogisticRegression(), # Wrap Logistic Regression
    require_dense=[True, True] # Require dense to speed up our process (we have
    ↪ sparse matrices, especially for tf-idf)
)

# Fit the training data and predict on test set for tfidf
logisticClassifier.fit(X_train_tfidfVectorised, y_train)
y_predLogisticTfidf = logisticClassifier.predict(X_test_tfidfVectorised)

# Same for GloVe
logisticClassifier.fit(X_train_GloVeVectorisedOutNA,
    ↪ y_trainGloVeVectorisedOutNA)
y_predLogisticGloVe = logisticClassifier.predict(X_test_GloVeVectorised)

# Save the results:
# Save the results for Logistic GloVe
with open("pickledFiles/y_predLogisticGloVe.pkl", "wb") as file:
    pickle.dump(y_predLogisticGloVe, file)

```

```
# Save the results for Logistic tfidf
with open("pickledFiles/y_predLogisticTfidf.pkl", "wb") as file:
    pickle.dump(y_predLogisticTfidf, file)
```

Report Now that we have computed all the baselines, we should evaluate them. Typically, we present the confusion matrix alongside the accuracy, precision, recall and F1 score. Unfortunately, this is not feasible in the context of multilabel classification. Indeed, a confusion matrix with all crossings would have 2^{27} rows, which is too large, and an aggregated confusion matrix with all labels would be too misleading.

In terms of accuracy, we cannot say that a combination of genres is incorrect just because one of the seven is missing, nor can we say that one has been predicted despite not being present. However, we can compare the accuracy of the predictions for each label. Ultimately, we will conduct this analysis:

- Per label: Accuracy, F1-Score, Precision, Recall
- Per rows: Jaccard similarity, Hamming Similarity, label accuracy product.

Here, we will not reexplain what accuracy, F1-Score, Precision and Recall are, but we will motivate the usages of the aggregation we will use.

First, the Jaccard similarity is computed as $J(Y, \hat{Y}) = \frac{\#Y \cap \hat{Y}}{\#Y \cup \hat{Y}}$, where $\hat{Y}$ is the predicted genres and Y the real genres. This metric is valued to 1 if the sets are identical, 0 if they are disjoint. It is convenient as it penalises both adding unnecessary genres, but also the absence of some. To get a value for the whole test set, we will average the row Jaccard similarity.

Second, the Hamming “Similarity” that we will define as: $H(Y, \hat{Y}) = 1 - \frac{H(Y, \hat{Y})}{\#Y}$, where H is the Hamming distance measuring the number of differences. This metric is valued at 1 if there is no difference between the vectors and at 0 if all is different. The difference with the Jaccard similarity is that it is sensitive to the number of 0, i.e. the sparser the vector, the more biased the result. As for Jaccard, we will take the average.

Finally, we will proceed with the product of accuracies per label. In a sense, if I can predict with an accuracy of 0.9 the genre **Action** and 0.7 the genre **Fantasy**, and ..., then by making the product, we would take into account all the errors. Unfortunately, it could be less meaningful compared to the other metrics, as it will quickly collapse to 0 considering the number of labels to predict (example: 90% of accuracy over all labels $\rightarrow 0.9^{27} \approx 0.0581$).

Let first proceed with the classic metric per labels.

```
[36]: # Function to compute the metrics
def computeLabelMetrics(y_true:np.ndarray[int], y_pred:np.ndarray[int],
    classes_:list[str]) -> pd.DataFrame:
    # Initialise the dataframe
    genreMetrics = pd.DataFrame(columns=["Genre", "Accuracy", "Precision",
    "Recall", "F1-Score"])

    # If it is a sparse matrix, convert it back into array
    if not isinstance(y_pred, np.ndarray):
        y_pred = y_pred.toarray()
```

```

# For each label
for label in range(y_true.shape[1]):
    # Compute the metrics
    labelMetric = [
        classes_[label],
        accuracy_score(y_true[:,label], y_pred[:,label]),
        precision_score(y_true[:,label], y_pred[:,label], zero_division=np.
↪nan),
        recall_score(y_true[:,label], y_pred[:,label]),
        f1_score(y_true[:,label], y_pred[:,label])
    ]
    # Register it
    genreMetrics.loc[label] = labelMetric

# Return the dataframe
return genreMetrics

# Compute the metrics for TFIDF over the 3 classifiers
labelMetricsBayesTfidf = computeLabelMetrics(y_test, y_predBayesTfidf, classes_)
labelMetricsLogisticTfidf = computeLabelMetrics(y_test, y_predLogisticTfidf, ↪
↪classes_)
labelMetricsEmbeddingTfidf = computeLabelMetrics(y_test, y_predEmbeddingTfidf, ↪
↪classes_)

# Compute the metrics for GloVe over the two classifiers
labelMetricsLogisticGloVe = computeLabelMetrics(y_test, y_predLogisticGloVe, ↪
↪classes_)
labelMetricsEmbeddingGloVe = computeLabelMetrics(y_test, y_predEmbeddingGloVe, ↪
↪classes_)

```

Let's take a look at the dataframes:

```
[37]: labelMetricsEmbeddingTfidf
```

```
[37]:
```

	Genre	Accuracy	Precision	Recall	F1-Score
0	Action	0.691736	NaN	0.0	0.0
1	Adult	0.870248	NaN	0.0	0.0
2	Adventure	0.728926	NaN	0.0	0.0
3	Comedy	0.659504	NaN	0.0	0.0
4	Drama	0.694215	NaN	0.0	0.0
5	Ecchi	0.952066	NaN	0.0	0.0
6	Fantasy	0.485950	NaN	0.0	0.0
7	Harem	0.826446	NaN	0.0	0.0
8	Historical	0.943802	NaN	0.0	0.0
9	Horror	0.965289	NaN	0.0	0.0
10	Josei	0.912397	NaN	0.0	0.0

11	Martial arts	0.909917	NaN	0.0	0.0
12	Mature	0.875207	NaN	0.0	0.0
13	Mystery	0.907438	NaN	0.0	0.0
14	Psychological	0.928099	NaN	0.0	0.0
15	Romance	0.378512	NaN	0.0	0.0
16	School life	0.856198	NaN	0.0	0.0
17	Sci-fi	0.920661	NaN	0.0	0.0
18	Seinen	0.949587	NaN	0.0	0.0
19	Shoujo	0.923967	NaN	0.0	0.0
20	Shounen	0.923967	NaN	0.0	0.0
21	Slice of life	0.817355	NaN	0.0	0.0
22	Supernatural	0.850413	NaN	0.0	0.0
23	Tragedy	0.909917	NaN	0.0	0.0
24	Xuanhuan	0.904132	NaN	0.0	0.0
25	Yaoi	0.802479	NaN	0.0	0.0
26	Yuri	0.968595	NaN	0.0	0.0

When we observe the data frame for the embedding classifier, we first notice that the recall is 0 and the precision is undefined. This is because our classifier classified everything as not having labels. Even after trying different optimisations in the development set by modifying the optimised metric (from accuracy to recall, F1 score [weighted, micro and macro] and Hamming similarity), we were unable to resolve this issue. Fortunately, this is not the case with the Bayes and Logistic classifiers.

```
[38]: labelMetricsBayesTfidf
```

```
[38]:
```

	Genre	Accuracy	Precision	Recall	F1-Score
0	Action	0.792562	0.724265	0.528150	0.610853
1	Adult	0.870248	NaN	0.000000	0.000000
2	Adventure	0.798347	0.690909	0.463415	0.554745
3	Comedy	0.666942	0.569231	0.089806	0.155136
4	Drama	0.693388	0.491228	0.075676	0.131148
5	Ecchi	0.952066	NaN	0.000000	0.000000
6	Fantasy	0.717355	0.701729	0.782958	0.740122
7	Harem	0.834711	0.666667	0.095238	0.166667
8	Historical	0.943802	NaN	0.000000	0.000000
9	Horror	0.965289	NaN	0.000000	0.000000
10	Josei	0.912397	NaN	0.000000	0.000000
11	Martial arts	0.909091	0.428571	0.027523	0.051724
12	Mature	0.875207	NaN	0.000000	0.000000
13	Mystery	0.909917	1.000000	0.026786	0.052174
14	Psychological	0.928099	NaN	0.000000	0.000000
15	Romance	0.731405	0.740157	0.875000	0.801950
16	School life	0.870248	0.904762	0.109195	0.194872
17	Sci-fi	0.921488	0.666667	0.020833	0.040404
18	Seinen	0.949587	NaN	0.000000	0.000000
19	Shoujo	0.924793	1.000000	0.010870	0.021505
20	Shounen	0.923967	NaN	0.000000	0.000000
21	Slice of life	0.818182	1.000000	0.004525	0.009009

22	Supernatural	0.849587	0.000000	0.000000	0.000000
23	Tragedy	0.909917	NaN	0.000000	0.000000
24	Xuanhuan	0.919835	0.913043	0.181034	0.302158
25	Yaoi	0.823967	0.795455	0.146444	0.247350
26	Yuri	0.968595	NaN	0.000000	0.000000

Here, we can see that the recall is very poor (nearly always close to 0); this indicates that either true positives are never predicted, or a lot of false negatives are occurring. Considering the high accuracy across the board, it is probably the former. The precision seems fine most of the time, as does the weighted F1-score.

Further analysis will involve investigating the results for all the datasets on a chart.

```
[39]: # Preprocess the dataframe for the plot
labelMetrics = pd.concat([
    labelMetricsBayesTfidf,
    labelMetricsLogisticTfidf,
    labelMetricsEmbeddingTfidf,
    labelMetricsLogisticGloVe,
    labelMetricsEmbeddingGloVe
], keys= [
    "Bayes TFIDF",
    "Logistic TFIDF",
    "Embedding TFIDF",
    "Logistic GloVe",
    "Embedding GloVe"
]).reset_index(level=0)

# Reset index before the plot
labelMetrics.rename(columns={"level_0": "source"}, inplace=True)
labelMetrics.reset_index(inplace=True)

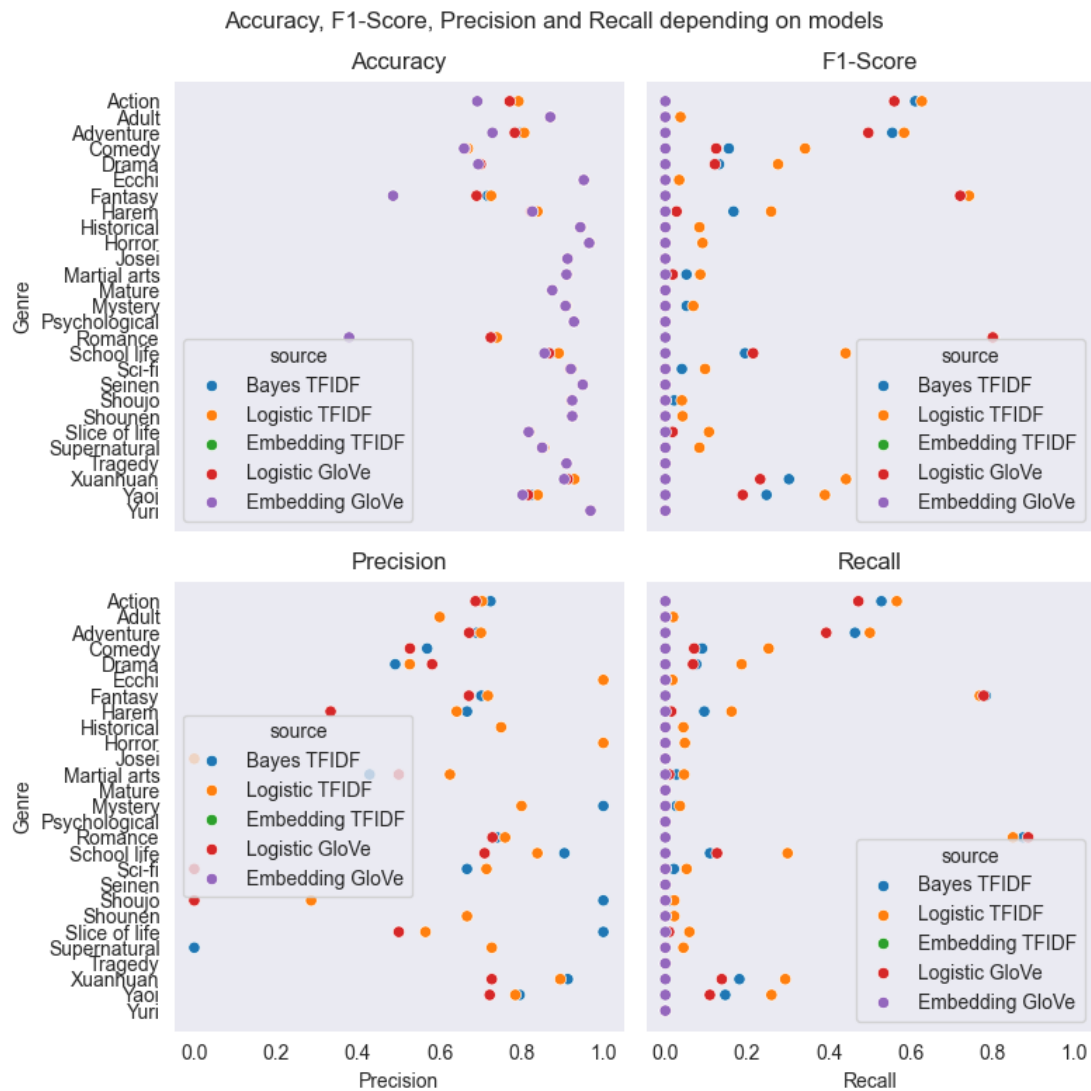
# Make a plot, adapted from a source on GeeksForGeeks
plt.close("all")

# Subplots
figure, axes = plt.subplots(2, 2, sharey=True, figsize=(8,8), sharex=True)
figure.suptitle('Accuracy, F1-Score, Precision and Recall depending on models')

# Scatterplot for all the metrics
sns.scatterplot(labelMetrics, y = "Genre", x="Accuracy", hue="source",
    ↪ax=axes[0,0])
sns.scatterplot(labelMetrics, y = "Genre", x="F1-Score", hue="source",
    ↪ax=axes[0,1])
sns.scatterplot(labelMetrics, y = "Genre", x="Precision", hue="source",
    ↪ax=axes[1,0])
sns.scatterplot(labelMetrics, y = "Genre", x="Recall", hue="source",
    ↪ax=axes[1,1])
```

```
# Set the titles
axes[0,0].set_title("Accuracy")
axes[0,1].set_title("F1-Score")
axes[1,0].set_title("Precision")
axes[1,1].set_title("Recall")

plt.tight_layout()
plt.show()
```



In terms of accuracy, we observe this from the global results: - GloVe embedding and TF-IDF under the similarities classifier are not convincing. Indeed, even though the accuracy is high, this is solely because we correctly predicted the true negative, which is the majority. Ultimately, the

classifier predicted 0 for everyone, which is sufficient for high accuracy. - Bayes is generally quite good with an accuracy of over 70%, with Logistic being slightly better.

We do not observe anything outstanding for the F1 score (apart from the embedding classifier, which has a recall of 0 due to the previously mentioned issue). Nevertheless, unlike accuracy, the F1-score is not particularly high. This could be explained by the imbalance in our label distribution from the outset. We also observe that GloVe performs significantly worse than TFIDF, thus confirming again that our document embeddings are not effective (averaging the GloVe embeddings for all tokens caused the semantics to disappear).

In terms of precision and recall, the precision is average, but the recall is poor. This suggests that our model has difficulty correctly identifying true positives.

For further analysis, we will therefore omit the GloVe representation and use TFIDF instead. We will also omit the embedding model from our analysis, since it performs worse than a simple Bayes model.

Now let's compute our row metrics.

```
[40]: # Function to compute the metrics
def computeRowMetrics(X_unprocessed:np.ndarray[str], y_true:np.ndarray[int],
    ↪y_pred:np.ndarray[int], classes_:list[str]) -> pd.DataFrame:
    # Initialise the dataframe
    observationMetrics = pd.DataFrame(columns=["Description", "Genres",
    ↪"Predicted Genres", "Jaccard", "Hamming"])
    classes_ = np.array(classes_)

    # If it is a sparse matrix, convert it back into array
    if not isinstance(y_pred, np.ndarray):
        y_pred = y_pred.toarray()

    # Compute the metric for each observations
    for i, description in enumerate(X_unprocessed):
        rowMetric = [
            description,
            list(classes_[np.where(y_true[i,:])[0]]),
            list(classes_[np.where(y_pred[i,:])[0]]),
            jaccard_score(y_true[i,:], y_pred[i,:]),
            1 - hamming_loss(y_true[i,:], y_pred[i,:])
        ]

        # Register it
        observationMetrics.loc[i] = rowMetric

    # Return the dataframe
    return observationMetrics.sort_values("Jaccard", ascending = False)
```

```
[41]: rowMetricsBayesTFIDF = computeRowMetrics(np.ravel(X_test[:, :, 0]), y_test,
    ↪y_predBayesTfidf, classes_)
```

```
rowMetricsLogisticTFIDF = computeRowMetrics(np.ravel(X_test[:, :, 0]), y_test,
↪ y_predLogisticTfidf, classes_)
```

Now that we have computed the metrics, we can observe the distribution of Jaccard similarity and Hamming similarity:

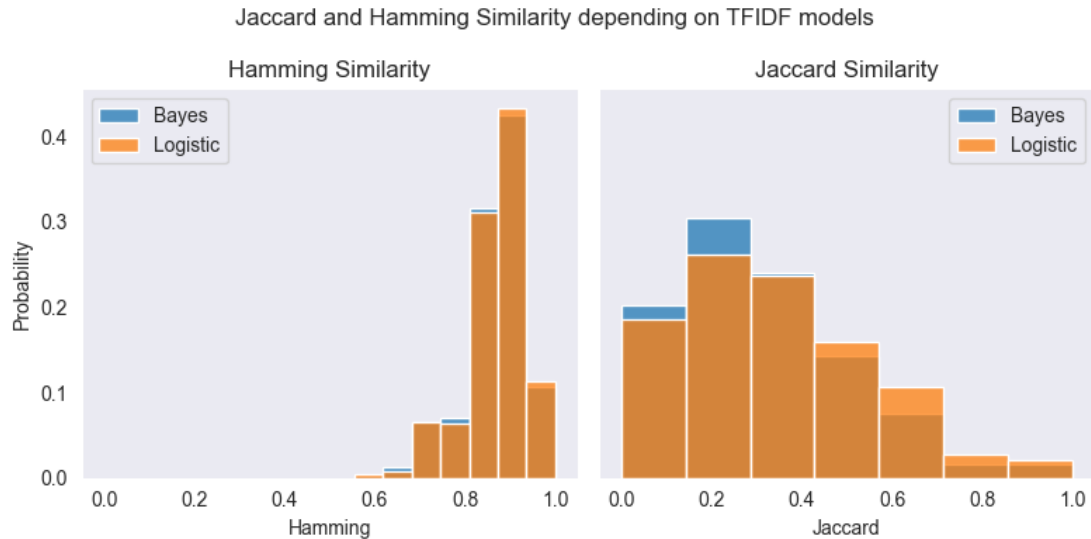
```
[42]: plt.close("all")

# Subplots
figure, axes = plt.subplots(1, 2, sharey=True, figsize=(8,4), sharex=True)
figure.suptitle('Jaccard and Hamming Similarity depending on TFIDF models')

# Scatterplot for all the metrics
sns.histplot(rowMetricsBayesTFIDF, x="Hamming", ax=axes[0], label="Bayes",
↪ bins=7, stat="probability")
sns.histplot(rowMetricsLogisticTFIDF, x="Hamming", ax=axes[0],
↪ label="Logistic", bins=7, stat="probability")
sns.histplot(rowMetricsBayesTFIDF, x="Jaccard", ax=axes[1], label="Bayes",
↪ bins=7, stat="probability")
sns.histplot(rowMetricsLogisticTFIDF, x="Jaccard", ax=axes[1],
↪ label="Logistic", bins=7, stat="probability")

# Set the titles
axes[0].set_title("Hamming Similarity")
axes[0].legend()
axes[1].set_title("Jaccard Similarity")
axes[1].legend()

# Plot the diagrams
plt.tight_layout()
plt.show()
```



As we stated earlier, the distribution clearly shows that Hamming similarity favours the absence of genres, while Jaccard similarity favours the correct presence of labels regardless of their sparse nature.

Regarding both models, we also observe that logistic regression is at least as precise as Bayes, which is good news. This shows that improving the model can improve the results for our data.

However, the Jaccard similarity is still low, as we will see shortly:

```
[43]: print("Bayes")
      rowMetricsBayesTFIDF.describe()
```

Bayes

```
[43]:
```

	Jaccard	Hamming
count	1210.000000	1210.000000
mean	0.299491	0.865963
std	0.214810	0.069648
min	0.000000	0.555556
25%	0.166667	0.814815
50%	0.250000	0.888889
75%	0.428571	0.925926
max	1.000000	1.000000

```
[44]: print("Logistic")
      rowMetricsLogisticTFIDF.describe()
```

Logistic

```
[44]:
```

	Jaccard	Hamming
count	1210.000000	1210.000000

mean	0.328350	0.869483
std	0.227840	0.069391
min	0.000000	0.555556
25%	0.166667	0.814815
50%	0.333333	0.888889
75%	0.500000	0.925926
max	1.000000	1.000000

Looking at these tables, it is clear that Hamming is not a good metric (the quartiles are too high and too close together, which is inconsistent with our earlier remarks on this metric). Regarding Jaccard, we observe a significant difference in the median between Bayes and Logistic; Logistic has higher second and third quartiles. The Logistic mean is also lower than the median, indicating that the distribution is skewed to the left. However, the mean and standard deviation are very close, which suggests a lack of precision. Moreover, the Jaccard index is only 32.8% on average, which is low, indicating overprediction or misprediction. In our case, it is probably misprediction.

For the final metric (product of accuracy), the new value is:

```
[45]: print(f"Bayes TFIDF Accuracy Product: {labelMetricsBayesTfidf['Accuracy'].
      ↪prod():.5f}")
      print(f"Logistic TFIDF Accuracy Product: {labelMetricsLogisticTfidf['Accuracy'].
      ↪prod():.5f}")
      print(f"Logistic/Bayes Accuracy Product: {labelMetricsLogisticTfidf['Accuracy'].
      ↪prod()/labelMetricsBayesTfidf['Accuracy'].prod():.5f}" )
```

Bayes TFIDF Accuracy Product: 0.01792

Logistic TFIDF Accuracy Product: 0.02012

Logistic/Bayes Accuracy Product: 1.12312

As predicted, this measure almost collapsed to 0%. Considering cumulative errors, the global accuracy of our system is 2% for Logistic and 1.79% for Bayes, representing a 12% improvement.

It is this result that motivated the transition to non-perfect match metrics, such as Jaccard similarity. Lower values are not dramatic as we do not expect perfect matches (otherwise it would be almost impossible computationally). In fact, depending on where the WN/LN description is found, the displayed genres may change, so it is not a big problem (as no one agrees on genres in the first place).

Before improving our system, let us analyse why some descriptions failed to be classified. We will analyse ten of them using Bayes and ten using Logistic. Let's start with Bayes.

```
[46]: rowMetricsBayesTFIDF.tail(10)
```

```
[46]:
```

	Description \
799	Song Luan transmigrated into a book called "Th...
803	As a person who transmigrated into a novel wri...
809	Peach farmer Toru discovers a sweet-smelling, ...
821	Back in 1996, the eldest daughter of Oldman Xi...
829	This book consists of a string of short storie...
830	The story of searching for a Tabi Pouch to put...

832 The distance of a perfect stand-in is the dist...
839 Bill: It referred to the devil. It was also th...
846 Hu Yu chased Fu Anli for three years. After be...
1209 One day, a virus capsule fell into a region an...

	Genres	Predicted Genres \
799	[Adult, Historical, Josei, Supernatural, Tragedy]	[Fantasy, Romance]
803	[Adventure, Drama, Martial arts, Mature, Psych...	[Romance]
809	[Fantasy, Yaoi]	[Romance]
821	[Drama, Fantasy]	[Romance]
829	[Slice of life]	[Fantasy, Romance]
830	[Drama, Tragedy]	[Fantasy, Romance]
832	[Drama, Yaoi]	[Romance]
839	[Drama, Yaoi]	[Romance]
846	[Adult, Comedy, Fantasy, School life, Supernat...	[Romance]
1209	[Drama, Mystery, Tragedy]	[Action, Fantasy]

	Jaccard	Hamming
799	0.0	0.740741
803	0.0	0.740741
809	0.0	0.888889
821	0.0	0.888889
829	0.0	0.888889
830	0.0	0.851852
832	0.0	0.888889
839	0.0	0.888889
846	0.0	0.740741
1209	0.0	0.814815

To begin with, each inferred genre in the 10 worst classifications were either **Fantasy**, **Romance** or **Action**. Considering the massive input in **Romance** and **Fantasy**, it seems that the Bayes classifier did, in fact, put almost everyone in these categories.

When reading the descriptions, we have the following observations: - [799]: We observe the words *Transmigrated*, *Wife*, *Male*, *Hug* and *Book*, which are usually signals of a classic **Romance/Isekai**, which are traditionally in a **Fantasy** setting. Here, the fantasy is replaced by the **Historical** and **Supernatural**. - [803]: The words *transmigrated*, *novel*, *villain*, *plot* and *story*, appearing in the description, are particularly important here. Indeed, they usually refer to the *I was reincarnated as the villainess and must survive* trope, which are usually tied to **Romance** and **Fantasy**. Here, there is the **Yaoi** tag, which is related to the **Romance** tag in some way. For the other tags, none were actually inferable from the description itself. - [809]: (What a weird description!) The key words here are: *Beautiful*, *Love* and *Heart*. Here, the attributed genres are directly related, unfortunately, we observe a confusion between the tags **Yaoi** and **Romance**. As for the fantasy tag, it does not really appear in some way (personally, I would have put **Supernatural** here, as it does more speak of the Peach Boy legend in Japanese lore). - [821]: When reading the description, we have dialogues, expressions and clinical terms (which are filtered out). In the end, we have the words *Female* and *Marry*, which are clearly biased toward **Romance**, even though the story is clearly not. (Sounds to be a good story, to be honest). - [829]: Here, as the description says, this is a compilation of

different stories, in consequence, the `Slice of Life` label was just not inferred, because here, it is not really the story which is told, but a description of the book, this makes discrimination difficult. I suppose that, for safety, the classifier did classify to the most frequent class. - [830]: Here, we do think the same happened, the classifier did make a default choice. - [832]: Here, the description does not provide any useful information related to our genres. - [839]: Again, we observe the confusion between `Romance` and `Yaoi`. We suppose that, in description, `Yaoi` and `Fantasy` are not really different, considering that most of the time, `Yaoi` tends to be more “explicit”, but they cannot put it in the eye of the large public. - [846]: (That’s weird, help! I do not want to analyse these anymore!) Same confusion, but failed to capture the `Supernatural/Fantasy` context, which could have been captured in the token *party*, which is usually used for `Guild/Adventure/Fantasy` context. - [1209]: Here, we observe a completely different context; we have the words *Zombie*, *Area*, *Virus*, *People* and *Fighting*, which clearly reflect `Action` (properly inferred), despite the genre not being in the attributed list. `Fantasy` could also have been a good guess here (despite being a bit surprising), so in this situation, we think it is more related to bad genre attributions/bad description.

Here, we observe a tendency among the errors: they are usually classic contexts and setups, but approached from an alternative point of view. This seems to destabilise our model. Additionally, some tags are very similar, so it would be a good idea to address this issue using rule mining or fuzzy links in the evaluation process. For example, if I match `Yaoi`, then I should also automatically match `Romance` at 70%.

Now, let us do the same for Logistic:

```
[47]: rowMetricsLogisticTFIDF.tail(10)
```

	Description \	Genres	Predicted Genres \
354	"I don't want to wash the dishes today." \n "I'l...	[Drama, Romance, Tragedy, Yaoi]	[]
1023	A letter to break off the engagement leads Lin...	[Action, Adult, Fantasy, Historical, Mature, Y...	[Romance]
344	Li Wei, who was hit by a speeding truck, came ...	[Action, Adventure, Fantasy, Harem, Mystery, P...	[Romance]
340	Her step-sister just died three months ago. No...	[Drama, Horror, Tragedy]	[Fantasy, Romance]
1029	B-class adventurer Gray found an orphan who so...	[Action, Adventure, Drama, Slice of life]	[Fantasy, Romance]
1032	20XX. Japan has become a super-aging society w...	[Action, Horror, School life, Shounen, Tragedy]	[Comedy, Romance]
1033	Four years ago, a mysterious air crash took aw...	[Mystery, Yaoi]	[]
325	The entire world bullies you, hates you, and ...	[Adventure, Drama, Horror, Romance, Supernatur...	[Action]
324	He is the eldest grandson of the Mu clan who h...	[Adult, Josei, Mature, Romance]	[]
1209	One day, a virus capsule fell into a region an...	[Drama, Mystery, Tragedy]	[Action]

	Jaccard	Hamming
354	0.0	0.851852
1023	0.0	0.740741
344	0.0	0.703704
340	0.0	0.814815
1029	0.0	0.777778
1032	0.0	0.740741
1033	0.0	0.925926
325	0.0	0.740741
324	0.0	0.851852
1209	0.0	0.851852

In general, we observe that **Fantasy** is inferred less frequently than with Bayes (perhaps because we violated the independence hypothesis with Bayes earlier...). Regarding the errors, we have the following:

-
-
-
-
-
- [1029]: We find the words *Adventurer* and *Class*, which are usually tied to the duo **Fantasy/Action**. Here, only the **Fantasy** was inferred; still, we do not have explanation (apart from the high number of **Romance** in the training data) for the presence of **Romance** tag here.
-
- [1033]: Here, we have no way to predict correctly the tag **Yaoi** (there is no sign in the description). Moreover, the story in itself is quite original (the context of airplane crash investigation is rare); this could explain why we could not predict tags at all.
-
- [324]: Here, we are faced with the classic trope: ‘I was saved by someone and fall in love with him’. This is classic in description, still, they base themselves into hidden “Implied Knowledge” (like 2nd Degré in French, or knowledge which is not explicit, because assumed known by everyone).

It is clear that the model is much less susceptible to classic confusions, but it does have problems with rare or unexpected combinations of genres. It also has problems with implied and underlying knowledge.

1.7.3 4.2 - Neural Model

In our case, we have already established that using pre-trained GloVe embeddings is not an appropriate approach. Additionally, TF-IDF does not capture the context behind the tokens. Based on the results of the logistic regression, we do not expect a better outcome from a classical neural network approach using TF-IDF. For this reason, we will start again from scratch, using BERT embedding alongside neural networks.

Let us begin by loading the models. Note that, we will take as a tokenizer the one directly coming with the model.

```
[48]: tokenizerBert = BertTokenizer.from_pretrained("bert-base-uncased")
      modelBert = BertModel.from_pretrained("bert-base-uncased")
      modelBert.to("cuda")
```

```
[48]: BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSdpaSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=768, out_features=3072, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=3072, out_features=768, bias=True)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
)
```

Now we will quickly test them:

```
[49]: # Tokenize using BERT
inputs = tokenizerBert(
    ["This is a test. And this is testing other things.", "Clearly peculiar,␣
    ↪isn't it?"],
    return_tensors="pt",    # PyTorch
    padding=True,
    truncation=True
)

# Pass through the model of Bert
with torch.no_grad():
    outputs = modelBert(**{k:v.to("cuda") for k, v in inputs.items()})

# Get the output
print(outputs.last_hidden_state[:,0,:].shape, outputs.last_hidden_state[:,0,:])␣
    ↪#[:,0,:] get us the embedding CLS, which is our word embedding

torch.Size([2, 768]) tensor([[ 0.2146,  0.2686, -0.3096, ..., -0.0706,  0.3809,
 0.8220],
                             [-0.1093,  0.2904, -0.4768, ..., -0.0608,  0.4758,
 0.6919]],
                             device='cuda:0')
```

This seems to work, we have 2 document embedding of size 768. Now let us transform our text into BERT embeddings.

```
[50]: def getBertEmbedding(Xset:np.ndarray[str], batchSize:int = 25) ->␣
    ↪list[list[float]]:

    # Initialise the list that will contains the embeddings
    Xset_embedded = []
    begin = time.time()

    # Put the Bert Model in evaluation mode
    modelBert.eval()
    # For each batch
    for i in range(0, (Xset.shape[0]/batchSize).__ceil__()):
        # Tokenize the batch
        XsetBertTokenized = tokenizerBert(
            # Get str to always have a valid sequence
            list(map(str, Xset[i*batchSize:(i+1)*batchSize,0])),
            return_tensors="pt",
            padding=True,
            truncation=True
        )
        # Print the advancement
        if i%10 == 0:
            print(f"Batch no. {i}/{(Xset.shape[0]/batchSize).__ceil__()} [{time.
            ↪time()-begin:.4f}s]")
```

```

        # Get the embedding
        with torch.no_grad():
            Xset_embedded.extend(modelBert(**{k:v.to("cuda") for k,v in XsetBertTokenized.items()})).last_hidden_state[:,0,:].tolist())

    return Xset_embedded

```

WARNING: This section can be long and is memory intensive, please use the next restore point instead

Now let's apply the function to get the embeddings:

```

[105]: # Embed and store for x_train
X_train_BertEmbedded = getBertEmbedding(X_train)
with open("pickledFiles/X_train_BertEmbedded.pkl", "wb") as file:
    pickle.dump(X_train_BertEmbedded, file)

```

```

Batch no. 0/351 [0.0440s]
Batch no. 10/351 [42.1358s]
Batch no. 20/351 [80.2049s]
Batch no. 30/351 [116.2258s]
Batch no. 40/351 [152.9954s]
Batch no. 50/351 [189.6008s]
Batch no. 60/351 [225.3508s]
Batch no. 70/351 [266.2898s]
Batch no. 80/351 [303.1241s]
Batch no. 90/351 [340.6601s]
Batch no. 100/351 [379.9119s]
Batch no. 110/351 [418.6431s]
Batch no. 120/351 [455.2897s]
Batch no. 130/351 [488.9063s]
Batch no. 140/351 [526.0096s]
Batch no. 150/351 [562.2970s]
Batch no. 160/351 [602.5130s]
Batch no. 170/351 [640.2159s]
Batch no. 180/351 [675.5029s]
Batch no. 190/351 [716.5175s]
Batch no. 200/351 [752.4084s]
Batch no. 210/351 [791.1448s]
Batch no. 220/351 [826.6645s]
Batch no. 230/351 [867.1187s]
Batch no. 240/351 [906.5725s]
Batch no. 250/351 [945.7907s]
Batch no. 260/351 [981.2867s]
Batch no. 270/351 [1020.0800s]
Batch no. 280/351 [1058.3542s]
Batch no. 290/351 [1095.8293s]
Batch no. 300/351 [1134.7388s]

```

```
Batch no. 310/351 [1173.3611s]
Batch no. 320/351 [1211.5717s]
Batch no. 330/351 [1247.7696s]
Batch no. 340/351 [1286.6023s]
Batch no. 350/351 [1326.8893s]
```

```
[106]: # Embed and store for developing set
X_dev_BertEmbedded = getBertEmbedding(X_dev)
with open("pickledFiles/X_dev_BertEmbedded.pkl", "wb") as file:
    pickle.dump(X_dev_BertEmbedded, file)
```

```
Batch no. 0/72 [0.0594s]
Batch no. 10/72 [36.6044s]
Batch no. 20/72 [72.8046s]
Batch no. 30/72 [110.3255s]
Batch no. 40/72 [146.5450s]
Batch no. 50/72 [182.1630s]
Batch no. 60/72 [218.9093s]
Batch no. 70/72 [256.0623s]
```

```
[107]: # Embed and store for test set
X_test_BertEmbedded = getBertEmbedding(X_test)
with open("pickledFiles/X_test_BertEmbedded.pkl", "wb") as file:
    pickle.dump(X_test_BertEmbedded, file)
```

```
Batch no. 0/49 [0.0456s]
Batch no. 10/49 [35.9369s]
Batch no. 20/49 [75.3157s]
Batch no. 30/49 [113.9096s]
Batch no. 40/49 [149.4133s]
```

1.7.4 Pickle restoration point

```
[51]: # Load for training
with open("pickledFiles/X_train_BertEmbedded.pkl", "rb") as file:
    X_train_BertEmbedded:list[list[float]] = pickle.load(file)

# Load for development
with open("pickledFiles/X_dev_BertEmbedded.pkl", "rb") as file:
    X_dev_BertEmbedded:list[list[float]] = pickle.load(file)

# Load for test
with open("pickledFiles/X_test_BertEmbedded.pkl", "rb") as file:
    X_test_BertEmbedded:list[list[float]] = pickle.load(file)
```

Just a test to know if the restoration worked:

```
[54]: print(X_train_BertEmbedded[0])
```

[0.09014085680246353, 0.2755882441997528, -0.4286854565143585,
0.05017227679491043, -0.2052931934595108, 0.033631838858127594,
0.2675405442714691, 0.7713221907615662, -0.4316599369049072,
-0.3154628276824951, -0.2862113416194916, -0.1158280000090599,
-0.23177826404571533, 0.5864198207855225, 0.4694485664367676,
0.3522716164588928, 0.12941165268421173, 0.5879907011985779,
-0.0034486709628254175, -0.29367896914482117, -0.13573169708251953,
0.3259432017803192, -0.012448015622794628, -0.22388339042663574,
0.3057641386985779, -0.20641325414180756, -0.08824871480464935,
-0.6144289970397949, -0.46756628155708313, 0.27753394842147827,
-0.0791308581829071, 0.4714924991130829, -0.24199238419532776,
-0.34756502509117126, 0.4172460734844208, -0.4126276671886444,
0.20028254389762878, 0.3006036877632141, 0.4776060879230499, 0.5354228019714355,
-0.035243794322013855, 0.14865480363368988, 0.47062671184539795,
-0.16678503155708313, -0.002738312818109989, 0.018882745876908302,
-3.817025661468506, 0.18465334177017212, -0.6666916608810425,
0.03794775530695915, -0.31615179777145386, -0.4693376123905182,
0.22404661774635315, 0.3257696330547333, -0.10136743634939194,
0.8435106873512268, 0.027537425979971886, 0.32076913118362427,
0.2308298498392105, -0.08930405974388123, 0.5658622980117798,
0.039842814207077026, -0.20311100780963898, -0.04725705832242966,
-0.11639219522476196, 0.21546940505504608, 0.1591368317604065,
0.5756856799125671, -0.7723830342292786, 0.5505660176277161, -0.349504292011261,
-0.282895565032959, -0.16561685502529144, -0.6189647316932678,
0.31389811635017395, -0.0948081910610199, -0.19857466220855713,
0.5459858775138855, -0.1546870619058609, -0.5502994656562805,
-0.25771573185920715, 0.33264869451522827, 0.4340525269508362,
0.3746335208415985, 0.46919506788253784, 0.29479801654815674,
0.09733997285366058, 0.020021215081214905, 0.30511215329170227,
0.274170458316803, -0.08546887338161469, -0.021939879283308983,
-0.3846028745174408, 0.29954931139945984, 1.0198694467544556,
-0.051862552762031555, 0.10854674875736237, -0.22609423100948334,
0.15784718096256256, 0.09313303232192993, -0.05000409856438637,
-0.17307014763355255, 0.4767623245716095, -0.8574589490890503,
-0.34351643919944763, 0.19299021363258362, 0.1092914566397667,
-0.6628184914588928, 0.2551700472831726, -1.8158427476882935,
0.25812193751335144, 0.508188784122467, 0.05425581336021423,
-0.3003392219543457, 0.40938466787338257, 0.07853980362415314,
0.036875490099191666, -0.20555761456489563, 0.20729303359985352,
0.1883348673582077, -0.3520652949810028, 0.48500189185142517,
0.2609216868877411, -0.1661679446697235, 0.20250993967056274,
0.3406166732311249, 0.20816174149513245, 0.011389288119971752,
0.512008011341095, 0.2808023989200592, 0.11957316845655441, 0.3921501338481903,
-0.3138899505138397, -0.5932527780532837, -0.44853416085243225,
0.26378515362739563, 0.5068268179893494, -0.19839182496070862,
-0.19325585663318634, -0.08313840627670288, -0.5034732222557068,
-0.4405665695667267, -2.8193650245666504, -0.06826530396938324,
1.0682061910629272, 0.42231807112693787, -0.5143256187438965,

0.10493922978639603, 0.09654384851455688, 0.648192286491394, 0.0324670672416687,
-0.10864706337451935, 0.22146950662136078, -0.028439871966838837,
-0.17953218519687653, 0.1929095834493637, 0.056610107421875, 0.2474430352449417,
-0.40459251403808594, 0.5835945010185242, 0.31986624002456665,
0.2008306086063385, 0.09615398198366165, -0.23429907858371735,
-0.15637391805648804, 0.3143417537212372, 0.11639478802680969,
-0.016527581959962845, -0.33530884981155396, -0.245277538895607,
-0.009230189025402069, -0.5258655548095703, 0.6112023591995239,
-0.03669728338718414, -0.05921134725213051, 0.4754693806171417,
0.12975847721099854, 0.5542024970054626, 0.11685020476579666,
-0.03489568457007408, -0.509410560131073, -0.2143251895904541,
-0.049952052533626556, -0.21833132207393646, 0.4265730082988739,
-0.16158321499824524, 0.34989815950393677, -0.20493574440479279,
-0.06993689388036728, 0.25638043880462646, -0.24395109713077545,
-0.7401164770126343, 0.46855512261390686, -0.17592741549015045,
0.7086147665977478, 0.03896196186542511, 0.02986261621117592,
-0.5622262954711914, 0.2223391979932785, 0.11281782388687134,
-0.15854249894618988, 0.3342587649822235, 0.016939442604780197,
0.17775505781173706, -0.3430812656879425, 3.8045623302459717,
0.028350021690130234, 0.21074999868869781, 0.02344123087823391,
0.2780219614505768, -0.03784864768385887, 0.02150769904255867,
0.2883845567703247, -0.1878928691148758, -0.02167285420000553,
0.3510642349720001, 0.4718366861343384, -0.1806711107492447,
-0.42393532395362854, -0.25389277935028076, -0.056939613074064255,
-0.3743095397949219, -0.12058714777231216, 0.3762308657169342,
0.2030545026063919, 0.40487048029899597, -0.3523382544517517,
0.2383900135755539, 0.46507206559181213, -1.4133926630020142,
0.5140088200569153, 0.035492654889822006, -0.16986030340194702,
-0.12098991125822067, -0.5386533737182617, 0.3971501588821411,
0.1839241236448288, 0.20835138857364655, -0.07552765309810638,
0.041362810879945755, -0.2104378640651703, 0.19310468435287476,
-0.2474745512008667, 0.11314047873020172, -0.4459867477416992,
-0.05142796412110329, 0.19608429074287415, -0.6568122506141663,
-0.060791123658418655, 0.40488165616989136, -0.06401896476745605,
0.28556838631629944, -0.35353076457977295, -0.19124968349933624,
-0.20803293585777283, 0.18621978163719177, 0.24891646206378937,
0.1370857208967209, -0.24916651844978333, 0.08337104320526123,
-0.3817707300186157, 0.09610692411661148, 0.4306206703186035,
0.013901435770094395, -0.8941095471382141, -0.3706829845905304,
-0.47117897868156433, -0.08757714927196503, -0.15795515477657318,
-0.007174638565629721, 0.1534755378961563, -0.5281462669372559,
-0.4380286633968353, -2.407550096511841, -0.10507938265800476,
0.08390713483095169, 0.239674374461174, 0.000890673662070185,
-0.4284069240093231, -0.45801812410354614, 0.07223028689622879,
0.22009670734405518, -0.32528015971183777, 0.41745954751968384,
0.21973012387752533, -0.1745038479566574, 0.3991933763027191,
-0.3307741582393646, 0.30390793085098267, 0.18696454167366028,
-0.08416444808244705, -0.013939707539975643, -0.05591747164726257,

-0.09461801499128342, 0.6826063990592957, -0.2701605558395386,
0.33974817395210266, 0.17738819122314453, -0.26881226897239685,
-0.3709656298160553, -0.10208477824926376, 0.2833145260810852,
-0.014974851161241531, -0.1223931536078453, 0.060007624328136444,
0.12946604192256927, -0.14102184772491455, -0.29441818594932556,
-4.069441318511963, 0.36789852380752563, -0.3745826184749603,
-0.3970358967781067, -0.42581936717033386, 0.10037731379270554,
0.5988159775733948, -0.16662375628948212, -0.359466552734375,
-0.02170509472489357, -0.1399916708469391, -0.32395753264427185,
0.06500786542892456, 0.1643010824918747, -0.13171173632144928,
0.3977494239807129, 0.7635530233383179, -0.005526241380721331,
0.223102867603302, 0.613765299320221, -0.2618548274040222, 0.20000503957271576,
0.16505511105060577, -0.08515410125255585, 0.32879921793937683,
0.18461452424526215, -0.5526759624481201, -0.057597555220127106,
-0.3437168598175049, -0.263839989900589, 0.02559165097773075,
-0.06798790395259857, -0.18304145336151123, -0.27757763862609863,
-0.12104673683643341, -0.4365452229976654, 0.004114068578928709,
0.24329526722431183, 0.41717565059661865, 0.06040501594543457,
-0.04841242730617523, 0.7439655661582947, -0.14263516664505005,
-0.03936802223324776, 0.5972874164581299, -0.2373688668012619,
-0.32472342252731323, -0.18383175134658813, 0.2541681230068207,
0.11380334198474884, -0.26916536688804626, 0.018017325550317764,
1.3954452276229858, -0.030190013349056244, 0.3091112971305847,
-0.08844311535358429, 0.6115025877952576, 0.3145805299282074,
0.019959459081292152, -0.27624353766441345, 1.0720551013946533,
-0.2632761597633362, 0.22242531180381775, -0.39945703744888306,
-0.14472024142742157, -0.6412495970726013, 0.20448902249336243,
-0.24233640730381012, -0.24670162796974182, 0.07557506114244461,
-0.4233311414718628, 0.7670401930809021, -0.2796838581562042,
-1.2864654064178467, 0.08812913298606873, 0.011617123149335384,
-0.32267674803733826, 0.25987181067466736, -0.1091536283493042,
0.2984452247619629, -0.4489862322807312, -0.01256552804261446,
-0.36496368050575256, 0.6551322937011719, -0.4895252287387848,
-0.30480843782424927, -0.09811151027679443, 0.47808265686035156,
-1.3482893705368042, 0.09774075448513031, -0.005669872742146254,
0.37295475602149963, -0.02193763107061386, 0.32945284247398376,
-0.0872611552476883, 0.416127473115921, 0.6153711676597595, -0.7554971575737,
-0.03595675528049469, -0.2798955738544464, 0.6458173394203186,
0.06870608776807785, -0.03980995714664459, -0.32933732867240906,
-0.5656441450119019, 0.39724498987197876, -0.3903770446777344,
0.5675509572029114, 0.2088087797164917, 0.020075799897313118,
-0.14652635157108307, 0.01819297857582569, -0.44447317719459534,
-0.13335758447647095, 0.8882341980934143, -0.27035248279571533,
-0.2254810482263565, 0.2762454152107239, 0.39112189412117004,
-0.1423105150461197, 0.14141282439231873, 0.0139309735968709,
-0.0661776214838028, -0.13958223164081573, -0.2083653062582016,
-0.03646587207913399, 0.022283069789409637, 0.17853569984436035,
0.027282198891043663, 0.195491224527359, -0.31005430221557617,

-0.2604139745235443, -0.2934038043022156, -0.5376074314117432,
-0.3388272225856781, -0.5435779690742493, -0.1024537906050682,
0.3731606900691986, 0.49688979983329773, 0.1079135462641716, 0.8546465039253235,
0.4853156507015228, -0.04963391274213791, 0.43413761258125305,
-0.49211257696151733, 0.7453057169914246, 0.13959580659866333,
-0.18684309720993042, -0.4808858335018158, 0.8088597655296326,
-0.3610248267650604, -0.2992086112499237, -0.2175082266330719,
-0.640066921710968, 0.585848867893219, -0.05958813428878784,
-0.37280571460723877, 0.4608701169490814, 0.006117425858974457,
-0.42521238327026367, 0.18227450549602509, 0.11778181791305542,
-1.9433319568634033, 0.2686386704444885, 0.41212281584739685,
0.13674116134643555, 0.15972676873207092, -0.3470782935619354,
-0.37396004796028137, 0.37040579319000244, -0.06866444647312164,
-0.003982742317020893, -0.34189245104789734, -0.0345834456384182,
-0.03097553364932537, -0.11079403758049011, 0.15262539684772491,
-0.34267091751098633, 0.34358957409858704, 0.11551535874605179,
-0.2594112753868103, -0.09073931723833084, -0.15416154265403748,
0.423526406288147, 0.4907767176628113, -0.1233632043004036, 0.03645504638552666,
-0.34293538331985474, -0.11184574663639069, 0.22382642328739166,
-0.12774159014225006, 0.8607595562934875, -0.03354477882385254,
-0.49158576130867004, -0.6879878640174866, -0.13993071019649506,
0.12447862327098846, 0.21369397640228271, 0.01948358118534088,
-0.20528346300125122, 0.58635014295578, 0.6318223476409912, -0.7743215560913086,
0.2782895565032959, 0.44401562213897705, -0.0437302328646183,
0.6888928413391113, 0.22656066715717316, -0.3964446187019348,
0.5751338601112366, 0.10968666523694992, -0.2852115333080292,
-0.561919093132019, -0.2425582855939865, -0.5660587549209595,
-0.45786190032958984, 0.021104881539940834, -0.17685401439666748,
0.07360938936471939, 0.1334480196237564, -0.0456063412129879,
-0.20904377102851868, -0.15345928072929382, -0.03309426084160805,
-0.30412614345550537, 0.09130380302667618, -0.717401385307312,
-0.5484954714775085, 0.06758932024240494, -0.2588106393814087,
-0.11398789286613464, 0.5550491213798523, 0.4103422462940216,
0.5427513122558594, -0.3406824767589569, -0.3670503795146942,
-0.6281763911247253, 0.10602288693189621, -0.15420874953269958,
-0.4227556884288788, 0.3359200358390808, -0.4945830702781677,
0.08596300333738327, -0.8609012961387634, -0.01876368373632431,
0.08583993464708328, -0.11812585592269897, 0.0166044682264328,
-0.4995245635509491, 0.06677498668432236, 0.1782645881175995,
-0.15289802849292755, -0.4926341474056244, -0.9286223649978638,
0.09594442695379257, -0.14874888956546783, 0.22379542887210846,
-0.2771856188774109, -0.36609044671058655, -0.020228203386068344,
0.2876232862472534, -0.12647253274917603, -0.3962494134902954,
0.5163542628288269, 0.15146507322788239, 0.6560678482055664,
0.33700698614120483, 0.36828601360321045, 0.23923809826374054,
0.09204769879579544, -0.5252049565315247, 0.21734003722667694,
-0.1036161482334137, 0.323388934135437, -0.046372346580028534,
-0.020851675420999527, -0.23533543944358826, -0.2657316029071808,

0.32809770107269287, -0.5832710862159729, 1.6507376432418823,
-0.04423815757036209, 0.09105399996042252, 0.5009218454360962,
0.7999869585037231, -0.5342816710472107, 0.15767981112003326,
-0.038975607603788376, -0.05913196876645088, 0.34375688433647156,
-0.538021981716156, 0.4788336455821991, -0.5352061986923218,
0.16350756585597992, 0.6655521392822266, 0.0453585647046566, 0.2864227294921875,
-0.7196617722511292, -0.6075719594955444, 0.11649231612682343,
-0.2193446159362793, 0.18650300800800323, 0.33238229155540466,
-0.029164735227823257, 0.22315514087677002, 0.1388174295425415,
0.3800082802772522, -0.5713331699371338, 0.22199316322803497,
0.7068548202514648, 0.30580979585647583, 0.03673062101006508,
0.11138385534286499, 0.7406866550445557, -0.21101731061935425,
-0.3806837499141693, -0.3553275465965271, -0.09307583421468735,
-0.25259026885032654, 0.10863123089075089, -0.03926372155547142,
-0.37158897519111633, 0.8249951004981995, -0.008655089884996414,
-0.14301200211048126, 0.6249669790267944, 0.04695479944348335,
-0.21886387467384338, 0.358481228351593, 0.4120267331600189,
0.011505324393510818, 0.043527718633413315, -0.33896055817604065,
0.11153888702392578, 0.19456151127815247, -0.40586793422698975,
0.204118013381958, -0.17395460605621338, -0.3689439296722412,
0.26108425855636597, -0.07395420223474503, 0.3526090681552887,
0.06250407546758652, -0.5719695687294006, -0.08056965470314026,
-0.17438198626041412, -0.4095916450023651, 0.3138161301612854,
0.12722833454608917, -0.6983128190040588, 0.6127257347106934,
0.2825404405593872, 0.2962436378002167, 0.224906787276268, 0.6215290427207947,
0.6078922152519226, 0.406937837600708, -0.08001414686441422,
-0.44547316431999207, -1.8512269258499146, 0.17787261307239532,
0.2530156672000885, 0.5426801443099976, -0.35977983474731445,
0.03186633065342903, -0.07961481064558029, -0.03930877894163132,
0.3544931411743164, 0.33351847529411316, 0.09253596514463425,
0.7958649396896362, 0.5204509496688843, 0.017913976684212685,
0.11990077048540115, -0.2800176441669464, 0.5185187458992004,
-0.12201272696256638, 0.32150882482528687, -0.04795705899596214,
0.13335020840168, 0.10347311943769455, -0.32471609115600586,
-0.28377291560173035, -0.435325026512146, 0.18696331977844238,
-0.4581124782562256, -0.06264810264110565, 0.45701849460601807,
0.5599071979522705, -0.014485865831375122, -0.03832753747701645,
-0.259678453207016, -0.7494716048240662, -0.09940986335277557,
-0.21120671927928925, -0.10367856919765472, 0.02914205752313137,
0.2967450022697449, -0.14988723397254944, -0.02200431004166603,
0.5385617613792419, -0.4382070302963257, 0.6234489679336548,
0.20936891436576843, 0.2105889916419983, 0.38664424419403076,
-0.4465070068836212, 0.513580322265625, 0.3970397710800171,
-0.04099831357598305, -0.019928915426135063, 0.11045773327350616,
-0.12136593461036682, 0.2909497618675232, 0.422047883272171,
0.43661338090896606, -0.38954755663871765, -0.13648495078086853,
-0.23924368619918823, -0.16899506747722626, -0.173023983836174,
-0.13927622139453888, -0.0911109670996666, -0.09412627667188644,

```
-0.2720680236816406, -0.17669500410556793, -0.33421626687049866,
0.262492835521698, -0.4748592674732208, -0.1473928689956665,
0.030323941260576248, 0.6274484395980835, 0.16897781193256378,
0.18694782257080078, 0.04481726884841919, -0.2327551394701004,
0.5972256660461426, 0.598906934261322, -0.1737903654575348,
-0.08685000240802765, 0.0023869595024734735, 0.1735047996044159,
0.09562642872333527, 0.3383251130580902, -5.274467945098877,
-0.14707522094249725, 0.07386144995689392, 0.004014275968074799,
-0.12062931805849075, -0.58915776014328, -0.1775856614112854,
-0.09952535480260849, -0.45864251255989075, -0.1065322756767273,
0.36045026779174805, 0.3935483992099762, -0.2571435272693634,
-0.30759957432746887, 0.6534921526908875, 0.5832481980323792]
```

Now that everything seems to be working, we can proceed with neural networks. Please note that multiple undisplayed iterations were made before achieving the following network. Here is the neural network that we will use:

```
[52]: class NeuralMultiLabelClassifier(nn.Module):
    # 768: dimension of the BERT embedding, 27: dimension of the label space
    def __init__(self, input_dim:int=768, num_labels:int=27, refine:bool =
    ↪False):
        super().__init__()

        # Main component to identify the features (input)
        self.feature = nn.Sequential(
            nn.Linear(input_dim, 512),
            nn.LayerNorm(512),
            nn.GELU(),
            nn.Dropout(0.3)
        )

        # Output layer
        self.classifier = nn.Linear(512, num_labels)

        # Linear Correlation learning module + the weight we will derive in the
    ↪logits
        self.label_correlation = nn.Sequential(
            nn.Linear(num_labels, num_labels, bias=False),
            nn.Tanh()
        )
        self.corr_gate = nn.Parameter(torch.tensor(0.5))

        # Refining layer (is active only if correlation is not). Used to
    ↪capture higher correlation relationships between labels
        self.addRefine = refine
        self.refine = nn.Sequential(
            nn.Linear(512 + num_labels, 256),
            nn.GELU(),
```

```

        nn.Linear(256, num_labels)
    )

    # Feed forward function
    def forward(self, x):
        # Get the features
        h = self.feature(x)
        # Classify
        logits = self.classifier(h)

        # Add the correlation between the labels
        if not self.addRefine:
            logits = logits + self.corr_gate*self.label_correlation(logits)
        else:
            logits = logits + self.refine(torch.cat([h, logits], dim=1))
        return logits

```

Now, we must explain and justify our architectural design. The architecture itself is divided into three consecutive parts. - **The Feature layer:** Constitute the hidden layer in order to learn the features associated with each token. It makes use of three main parts: a dense layer to learn the components; a normalisation layer to stabilise the model and ensure convergence; and a GELU (Gaussian Error Linear Unit) activation function, which apparently improves performance in Transformer contexts, as suggested [here](#) (BERT uses a Transformer, so it makes sense to use this activation function here). It also uses a 30% dropout layer to avoid overfitting (this dropout is used because our model is relatively simple, and the standard range of values is between 20% and 50%, as seen [here](#)). In a sense, it could be a tuneable parameter, but this will not be exploited here. - **The Classifier layer:** This layer connects the networks to the 27 output labels. We do not perform softmax here, as this would eliminate the ability to predict multiple labels simultaneously. In order to interpret the results, we must first find the optimal threshold at which a genre is considered to have been acquired. - **(Choice 1) The Correlation Layer:** Passing through the first two layers ensured that we could use the correlation between tokens. However, this does not take into account the correlation between labels. This is why we apply this layer: to learn the correlation between the label and apply corrections to the results. Note that we use Tanh afterwards for the same reason of normalisation (here, between -1 and 1 to respect the meaning of the correlation); also, the potential noise created by this layer is mitigated using a learnable parameter. Basically, this layer says: “If I increase A, then I should also increase/decrease B”. - **(Choice 2) The Refinement Layer:** By using a refinement layer instead of a correlation layer, we are not limited to direct correlation; we can also investigate nonlinear, deeper patterns, such as conditional or multi-correlated patterns. Therefore, we use a dense layer with dimensions of 512+27 to take the features and labels into account.

In the end, we have an architecture like this: 1. **Raw documents** 2. **Bert Embeddings** 3. **Feature Layer** 4. **Classifier Layer** 5. **Correlation | Refinement Layer** 6. **Sigmoid (prediction)** 7. **Thresholds (to go back to binary)**

For the tuneable parameters, we have the dropout rate, which we will not be adjusted, and, more importantly, the thresholds. Each output may have a different distribution. Consequently, we cannot set a global threshold such as *IF pred>0.5 THEN PREDICT 1*; this would not make sense and would not take into account the label imbalance, which is probably the biggest challenge here,

as we will explain.

Now that we explained the architecture, let us normalise our embeddings:

```
[53]: # Normalise the Bert Embedding
X_train_BertEmbeddedNormalised = nn.functional.normalize(torch.
    ↪tensor(X_train_BertEmbedded))
X_dev_BertEmbeddedNormalised = nn.functional.normalize(torch.
    ↪tensor(X_dev_BertEmbedded))
X_test_BertEmbeddedNormalised = nn.functional.normalize(torch.
    ↪tensor(X_test_BertEmbedded))
```

Now, we proceed with the training loop. Here, we will do the training for both model with correlation layer and refinement layer.

```
[57]: def trainNN(X_train:torch.tensor, Y_train:torch.tensor, batchSize:int = 100,
    ↪epochs:int=50, refine:bool=True, seed:int=0) -> NeuralMultiLabelClassifier:
    # Setup the seed of torch for reproducibility
    torch.manual_seed(seed)

    # Because the labels are unbalanced, use the Weighted BCEWithLogitLoss so
    ↪that we can add a higher weight to rare labels
    positiveCount = Y_train.sum(axis=0)
    negativeCount = Y_train.shape[0] - positiveCount
    positiveWeight = negativeCount / positiveCount
    criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(positiveWeight))

    # Initialise the Neural Model
    neuralModel = NeuralMultiLabelClassifier(refine=refine)

    # Use classic Adam with classic learning rate
    optimizer = torch.optim.Adam(neuralModel.parameters(), lr=1e-4)

    # Start the training loop
    begin = time.time()
    neuralModel.train()
    for epoch in range(epochs):
        # Inform of the advancement
        if epoch%10 == 0:
            print(f"[{time.time()-begin:.3f}s] {epoch=}")

        # Permute the batches at each epoch to avoid Overfitting. The random
        ↪seed is set to epoch to keep determinism
        rng = np.random.RandomState(epoch)
        permutation = rng.permutation(np.arange(X_train.shape[0]))
        X_epoch = X_train[permutation]
        Y_epoch = Y_train[permutation]
```

```

        # For each batch
        for batch in range(0, (X_train.shape[0]/batchSize).__ceil__()):
            # Reinitialise gradient
            optimizer.zero_grad()

            # Compute logits and loss for the current batch
            logits = neuralModel(torch.tensor(X_epoch[batch*batchSize:
↪(batch+1)*batchSize,:]))
            loss = criterion(logits, torch.tensor(Y_epoch[batch*batchSize:
↪(batch+1)*batchSize])).float()

            # Back propagation
            loss.backward()
            optimizer.step()

        # Return the model
        return neuralModel

# Get a model without refinement
print("[NORMAL TRAINING]")
neuralModelBaseline = trainNN(X_train_BertEmbeddedNormalised, y_train, ↪
↪refine=False)

# Get a model with refinement
print("[REFINE TRAINING]")
refinedNeuralModel = trainNN(X_train_BertEmbeddedNormalised, y_train, ↪
↪refine=True)

```

[NORMAL TRAINING]

[0.000s] epoch=0

C:\Users\arnol\AppData\Local\Temp\ipykernel_17968\2601788781.py:37: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).

```

    logits =
neuralModel(torch.tensor(X_epoch[batch*batchSize:(batch+1)*batchSize,:]))

```

[3.996s] epoch=10

[7.625s] epoch=20

[11.298s] epoch=30

[15.062s] epoch=40

[REFINE TRAINING]

[0.000s] epoch=0

[4.521s] epoch=10

[9.037s] epoch=20

[13.273s] epoch=30

[17.767s] epoch=40

```
[58]: # Save the results
torch.save(neuralModelBaseline, "pickledFiles/neuralModelBaseline.pth")
torch.save(refinedNeuralModel, "pickledFiles/refinedNeuralModel.pth")
```

As mentioned earlier, there are two main challenges in this classification task: the high correlation between the labels, which has already been addressed in the architecture, and the imbalance in the labels, which was observed at the start of the project.

To address the latter, we use a specific loss function: Weighted Binary Cross Entropy with Logits, as suggested [here](#). We use an inverse weight on positive labels to prioritise rare labels over frequent ones (to avoid making the same mistake as with the GloVe embedding earlier, where rare labels were never predicted).

Regarding the optimiser, we opted for the standard Adam with a standard learning rate.

To avoid overfitting, we only trained the model for 50 epochs, which seems appropriate for a simple model like this one, and permuted the order of the batches at every new epoch to prevent the model from learning an order or pattern within the batches. The seed of the permutation is set to the epoch value to maintain reproducibility.

Now that this is complete, we can discuss one of the most important aspects: threshold tuning.

```
[59]: def tuneOutputThresholds(y_true:np.ndarray[int], y_pred:np.ndarray[int],
    ↪classes_:list[str], precision:float=10**4) -> np.ndarray[float]:
    thresholds = []
    begin = time.time()

    # For each label
    for label in range(y_true.shape[1]):
        # Initialise the threshold and MacroF1 (we use macroF1 because of class
    ↪imbalances)
        bestThreshold, bestF1 = 0.5, 0

        # For each threshold
        for threshold in np.linspace(0, 1, precision):
            # Make the prediction based on the threshold
            preds = (y_pred[:, label] > threshold).astype(int)

            # Compute the MacroF1 for the given label
            f1 = f1_score(y_true[:,label], preds, average="macro")

            # If it is the best F1-Score, update the threshold
            if f1 > bestF1:
                bestF1 = f1
                bestThreshold = threshold

        # Register the final threshold
        thresholds.append(bestThreshold)
```

```

    # Inform the user
    print(f"[{time.time()-begin:.4f}s] Label {label+1}/{y_true.shape[1]}_
↳({classes_[label]}) done. Best threshold: {bestThreshold:.6f}")

    # Return the thresholds as numpy arrays
    return np.array(thresholds)

```

For threshold tuning, we simply find the thresholds (with 0.0001 precision) that maximise the F1 score for the labels in question. Each label has its own threshold to match its distribution.

Note that we do not use accuracy for tuning. Indeed, when examining GloVe, we already saw that accuracy was not a good metric. From this observation, we use the F1 score, which is more resistant to class imbalance. Initially, we tried to maximise the Jaccard index, but this proved counterproductive as it penalised too many false positives for rare labels, leading to no predictions for rare labels.

We can now compute the thresholds for the two versions of our neural network.

Warning: this part can be long, go to the next pickle restoration point instead

```

[361]: # Fine tune the thresholds for the normal training
neuralModelBaseline.eval()
with torch.no_grad():
    probs = nn.functional.
↳sigmoid(neuralModelBaseline(X_dev_BertEmbeddedNormalised))
    thresholdsBaseline = tuneOutputThresholds(y_dev, probs.detach()).
↳numpy(),classes_, precision=10**4)

```

```

[11.2358s] Label 1/27 (Action) done. Best threshold: 0.673467
[21.9861s] Label 2/27 (Adult) done. Best threshold: 0.767977
[32.9862s] Label 3/27 (Adventure) done. Best threshold: 0.568457
[43.8445s] Label 4/27 (Comedy) done. Best threshold: 0.517252
[54.6725s] Label 5/27 (Drama) done. Best threshold: 0.591859
[65.1940s] Label 6/27 (Ecchi) done. Best threshold: 0.718772
[76.1462s] Label 7/27 (Fantasy) done. Best threshold: 0.508851
[86.9281s] Label 8/27 (Harem) done. Best threshold: 0.655466
[97.5822s] Label 9/27 (Historical) done. Best threshold: 0.855786
[108.0922s] Label 10/27 (Horror) done. Best threshold: 0.452745
[118.8581s] Label 11/27 (Josei) done. Best threshold: 0.730673
[129.5006s] Label 12/27 (Martial arts) done. Best threshold: 0.703870
[140.2125s] Label 13/27 (Mature) done. Best threshold: 0.604960
[150.8045s] Label 14/27 (Mystery) done. Best threshold: 0.555256
[161.4350s] Label 15/27 (Psychological) done. Best threshold: 0.815782
[172.4360s] Label 16/27 (Romance) done. Best threshold: 0.433043
[183.1869s] Label 17/27 (School life) done. Best threshold: 0.756176
[193.8211s] Label 18/27 (Sci-fi) done. Best threshold: 0.723072
[204.4219s] Label 19/27 (Seinen) done. Best threshold: 0.829783
[215.0413s] Label 20/27 (Shoujo) done. Best threshold: 0.610661
[225.7420s] Label 21/27 (Shounen) done. Best threshold: 0.794479

```



```
[236.4930s] Label 22/27 (Slice of life) done. Best threshold: 0.613161
[247.1600s] Label 23/27 (Supernatural) done. Best threshold: 0.578558
[257.7428s] Label 24/27 (Tragedy) done. Best threshold: 0.628263
[268.4937s] Label 25/27 (Xuanhuan) done. Best threshold: 0.635464
[279.3946s] Label 26/27 (Yaoi) done. Best threshold: 0.575458
[289.8288s] Label 27/27 (Yuri) done. Best threshold: 0.733873
```

```
[362]: # Fine tune the threshold for the refined training
refinedNeuralModel.eval()
with torch.no_grad():
    probs = nn.functional.
    ↪sigmoid(refinedNeuralModel(X_dev_BertEmbeddedNormalised))
    thresholdsRefined = tuneOutputThresholds(y_dev, probs.detach().
    ↪numpy(),classes_, precision=10**4)
```

```
[11.0643s] Label 1/27 (Action) done. Best threshold: 0.684568
[21.8148s] Label 2/27 (Adult) done. Best threshold: 0.802280
[32.8010s] Label 3/27 (Adventure) done. Best threshold: 0.521652
[43.7029s] Label 4/27 (Comedy) done. Best threshold: 0.568357
[54.5183s] Label 5/27 (Drama) done. Best threshold: 0.564756
[65.0365s] Label 6/27 (Ecchi) done. Best threshold: 0.762176
[76.0537s] Label 7/27 (Fantasy) done. Best threshold: 0.440944
[86.8548s] Label 8/27 (Harem) done. Best threshold: 0.765077
[97.4909s] Label 9/27 (Historical) done. Best threshold: 0.898490
[107.9086s] Label 10/27 (Horror) done. Best threshold: 0.275928
[118.6578s] Label 11/27 (Josei) done. Best threshold: 0.707171
[129.3843s] Label 12/27 (Martial arts) done. Best threshold: 0.833283
[140.3948s] Label 13/27 (Mature) done. Best threshold: 0.628863
[151.0788s] Label 14/27 (Mystery) done. Best threshold: 0.509051
[161.8086s] Label 15/27 (Psychological) done. Best threshold: 0.865287
[172.7955s] Label 16/27 (Romance) done. Best threshold: 0.424742
[183.5749s] Label 17/27 (School life) done. Best threshold: 0.773577
[194.2109s] Label 18/27 (Sci-fi) done. Best threshold: 0.752575
[204.9153s] Label 19/27 (Seinen) done. Best threshold: 0.849085
[215.4995s] Label 20/27 (Shoujo) done. Best threshold: 0.610161
[226.1549s] Label 21/27 (Shounen) done. Best threshold: 0.760076
[237.3333s] Label 22/27 (Slice of life) done. Best threshold: 0.677968
[248.1509s] Label 23/27 (Supernatural) done. Best threshold: 0.679868
[258.8855s] Label 24/27 (Tragedy) done. Best threshold: 0.801580
[269.5381s] Label 25/27 (Xuanhuan) done. Best threshold: 0.766677
[280.4885s] Label 26/27 (Yaoi) done. Best threshold: 0.628263
[290.9882s] Label 27/27 (Yuri) done. Best threshold: 0.822882
```

```
[363]: # Save the thresholds
with open("pickledFiles/thresholdsBaseline.pkl", "wb") as file:
    pickle.dump(thresholdsBaseline, file)

with open("pickledFiles/thresholdsRefined.pkl", "wb") as file:
```

```
pkl.dump(thresholdsRefined, file)
```

1.7.5 Pickle Restoration Point

```
[54]: # Reload the thresholds of baseline
with open("pickledFiles/thresholdsBaseline.pkl", "rb") as file:
    thresholdsBaseline = pkl.load(file)

# Reload the thresholds of refined
with open("pickledFiles/thresholdsRefined.pkl", "rb") as file:
    thresholdsRefined = pkl.load(file)

# Reload the models
neuralModelBaseline = torch.load("pickledFiles/neuralModelBaseline.pth",
    ↪weights_only=False)
refinedNeuralModel = torch.load("pickledFiles/refinedNeuralModel.pth",
    ↪weights_only=False)
```

Even at this stage, it is clear that the thresholds differ significantly between labels and models. This confirms our hypothesis that the thresholds must be different for each label and not set to any defaults.

We can now make our predictions and save the results.

```
[55]: # Get the predictions for the normal model
neuralModelBaseline.eval()
with torch.no_grad():
    y_predNeuralModelBaseline = nn.functional.
    ↪sigmoid(neuralModelBaseline(X_test_BertEmbeddedNormalised)).detach().
    ↪numpy()>thresholdsBaseline
```

```
[56]: refinedNeuralModel.eval()
with torch.no_grad():
    y_predRefinedNeuralModel = nn.functional.
    ↪sigmoid(refinedNeuralModel(X_test_BertEmbeddedNormalised)).detach().
    ↪numpy()>thresholdsRefined
```

Now, we will compare to the other models for the Jaccard Similarity distribution:

```
[57]: rowMetricsNeuralBaseline = computeRowMetrics(X_test[:,0,0], y_test,
    ↪y_predNeuralModelBaseline, classes_)
rowMetricsNeuralRefined = computeRowMetrics(X_test[:,0,0], y_test,
    ↪y_predRefinedNeuralModel, classes_)
```

```
[58]: plt.close("all")

# Subplots
fig, ax = plt.subplots(1, 1, figsize=(8,4))
```

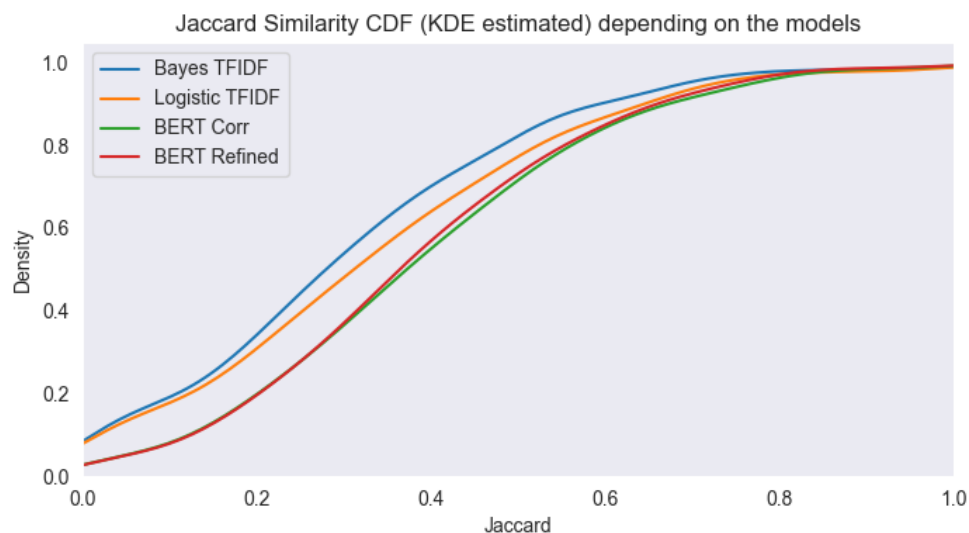
```

# Scatterplot for all the metrics
sns.kdeplot(rowMetricsBayesTFIDF, x="Jaccard", ax=ax, label="Bayes TFIDF",
            ↪cumulative=True)
sns.kdeplot(rowMetricsLogisticTFIDF, x="Jaccard", ax=ax, label="Logistic
            ↪TFIDF", cumulative=True)
sns.kdeplot(rowMetricsNeuralBaseline, x="Jaccard", ax=ax, label="BERT Corr",
            ↪cumulative=True)
sns.kdeplot(rowMetricsNeuralRefined, x="Jaccard", ax=ax, label="BERT Refined",
            ↪cumulative=True)

# Set the titles
ax.set_title("Jaccard Similarity CDF (KDE estimated) depending on the models")
ax.set_xlim(0,1)
ax.legend()

# Plot the diagrams
plt.show()

```



From the cumulative distribution function of the models created so far, we can see that Bayes and Logistic over TFIDF perform the worst overall. There is no significant difference between the two BERT methods; however, their performance is better than the baseline. Looking at the median Jaccard similarity, we see an improvement of 20.84% for BERT Corr compared to Logistic TFIDF. The difference is even more impressive at the first quartile, with improvements of 44.1% for both compared to Logistic.

Note: changing the seed sometimes resulted in BERT Refined performing better, so we would say there is not much difference between the two models.

We can also try visualising our embedding to see if the environment could be easily separated

depending on the labels. To do this, we will use TSNE, a dimensionality reduction technique that preserves distances between points, to observe a 768-dimensional vector in a 2-dimensional plane.

```
[59]: tsne = TSNE()
      embeddingTrainVisualisation = tsne.
      ↪fit_transform(X_train_BertEmbeddedNormalised, y_train)

[60]: # The colour were initially chosen by ChatGPT, but later modified to best suit ↵
      ↪our use-case
      genre_colours = {
          # Family 1 (Old & Evolution)
          'Action':      (139, 69, 19),
          'Martial arts': (139, 69, 19),
          'Shounen':      (139, 69, 19),
          'Historical':   (139, 69, 19),      # Saddle Brown
          'Xuanhuan':      (139, 69, 19),

          # Family 2 (Romance and associated)
          'Romance':      (255, 20, 147),
          'Ecchi':        (255, 20, 147),
          'Yuri':         (255, 20, 147),      # Deep Pink
          'Yaoi':         (255, 20, 147),
          'Adult':        (255, 20, 147),
          'Drama':        (255, 20, 147),
          'Harem':        (255, 20, 147),

          # Family 3 (Dark phenomenon)
          'Mature':       (0, 0, 0),
          'Tragedy':      (0, 0, 0),
          'Horror':       (0, 0, 0),          # Black
          'Mystery':      (0, 0, 0),
          'Psychological': (0, 0, 0),

          # Family 4 (Other worldly)
          'Adventure':    (0, 191, 255),
          'Fantasy':      (0, 191, 255),
          'Supernatural': (0, 191, 255),
          'Sci-fi':       (0, 191, 255),      # Deep Sky Blue

          # Family 5 (Day-to-day life)
          'Shoujo':       (255, 215, 0),      # Gold
          'School life':  (255, 215, 0),
          'Comedy':       (255, 215, 0),
          'Seinen':       (255, 215, 0),
          'Josei':        (255, 215, 0),
          'Slice of life': (255, 215, 0),
      }
```

```

def colourY(y_train:np.ndarray[int], classes_:np.ndarray[str], genre_colours:
    ↳dict[str,tuple[int]]) -> list[tuple[int,int,int]]:
    # Just asked chat gpt to define "coherent" colour for each genres
    # https://stackoverflow.com/questions/649454/
    ↳what-is-the-best-way-to-average-two-colors-that-define-a-linear-gradient

    colourValue = []
    for i in range(y_train.shape[0]):
        # Get the colours to the corresponding genres
        values = [genre_colours[genre] for genre in classes_[np.
    ↳where(y_train[i,:])[0]]]
        r, g, b = 0, 0, 0

        if len(values) == 0:
            colourValue.append((1,1,1,0))
            continue

        # Compute the Root Mean Squared colour (simple average would be to fade)
        for ri, gi, bi in values:
            r+=ri**2
            g+= gi**2
            b+=bi**2

        # Append a normalised value between 0 and 1
        colourValue.append([
            min(max(((r/len(values))*0.5)/255,0),1),
            min(max(((g/len(values))*0.5)/255,0),1),
            min(max(((b/len(values))*0.5)/255,0),1),
            1/len(values)
        ])
    return colourValue

# Plot the results

plt.close("all")
plt.figure(figsize=(8, 8))

# The legend logic was produced by ChatGPT (we had no clues on how to do this,
    ↳with matplotlib).
def normalize(rgb):
    return tuple(v / 255 for v in rgb)

legend_elements = []
labels = []

# Create the legend

```

```

for genre, color in genre_colours.items():
    legend_elements.append(
        Line2D(
            [0], [0],
            marker='o',
            linestyle='',
            markerfacecolor=normalize(color),
            markeredgecolor='black',
            markersize=8
        )
    )
    labels.append(genre)

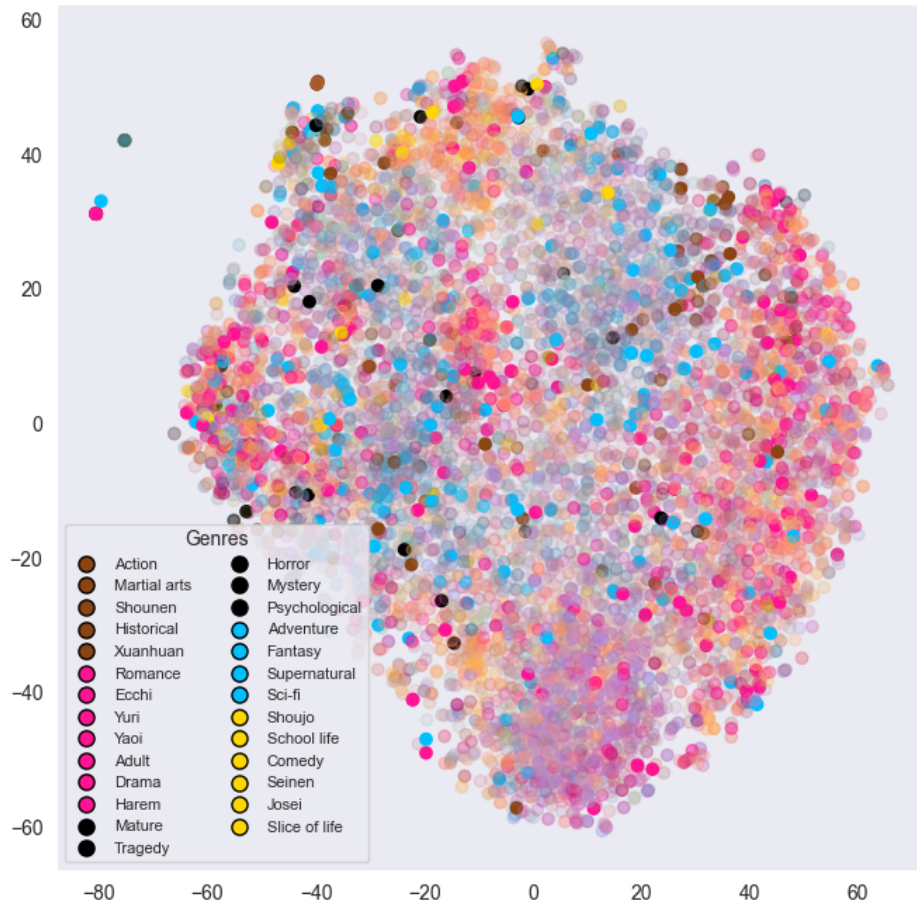
# Plot the points with the defined colour
plt.scatter(embeddingTrainVisualisation[:,0], embeddingTrainVisualisation[:,1],
            c=np.array(colourY(y_train, classes_, genre_colours)))
plt.title("The more a point is transparent, the more genres are associated with it.")
plt.suptitle("Average genre for each BERT embedding of the training set")

# Show the built legend
plt.legend(
    handles=legend_elements,
    labels=labels,
    title="Genres",
    loc='best',
    ncol=2,
    fontsize=8
)
# Plot the diagrams
plt.show()

```

Average genre for each BERT embedding of the training set

The more a point is transparent, the more genres are associated with it.



From this plot, we can clearly see that this task was not trivial. Some genres are heavily convoluted with others, some entries possess way more labels than others, and so on. Still, by choosing the colour wisely and putting a bonus on the number of genres, which is basically the same as rewarding ‘pure’ genres (only **Fantasy**, or only **Adventure**, and so on); we can capture some information:

1. The **Romance** & Co family is universal and does not tend towards a particular place.
2. The **Slice of Life** & Co family is also distributed evenly, except a blob higher in the sphere where it can be seen as “purer”. On the other side, it is also frequently associated with **Romance**, as seen in the very middle of the sphere.
3. The **Fantasy** family is fitted at the centre; this makes sense because **Fantasy** and co are frequently an environment associated to other genres. Such a frequent co-occurrence of genres are **Fantasy** and **Romance**.
4. Finally, **Martial Arts** and **Shounen** do emerge, but in a very small part.

1.8 6. Error Analysis & Interpretation

In this part, we will compare all the models in terms of performance and committed errors.

First, let's regroup all the results we had earlier:

Model/Metrics	Median Jaccard Similarity	Median Hamming Similarity	Accuracy Product
Bayes TFIDF	0.250	0.889	0.018
Logistic TFIDF	0.333	0.889	0.020
TFIDF Embedding Similarity	0	0.889	0.005
Logistic GloVe Embedding	0.250	0.889	0.016
GloVe Embedding Similarity	0	0.889	0.005
BERT Correlation	0.375	0.852	0.008
BERT Refined	0.3636	0.852	0.007

And the same for the genres:

```
[61]: # Preprocess the dataframe for the plot
labelMetrics = pd.concat([
    labelMetricsBayesTfidf,
    labelMetricsLogisticTfidf,
    labelMetricsEmbeddingTfidf,
    labelMetricsLogisticGloVe,
    labelMetricsEmbeddingGloVe,
    computeLabelMetrics(y_test, y_predNeuralModelBaseline, classes_),
    computeLabelMetrics(y_test, y_predRefinedNeuralModel, classes_)
], keys= [
    "Bayes TFIDF",
    "Logistic TFIDF",
    "Embedding TFIDF",
    "Logistic GloVe",
    "Embedding GloVe",
    "BERT Correlation",
    "BERT Refined"
]).reset_index(level=0)

# Reset index before the plot
labelMetrics.rename(columns={"level_0": "source"}, inplace=True)
labelMetrics.reset_index(inplace=True)

# Make a plot, adapted from a source on GeeksForGeeks
plt.close("all")

# Subplots
```



```

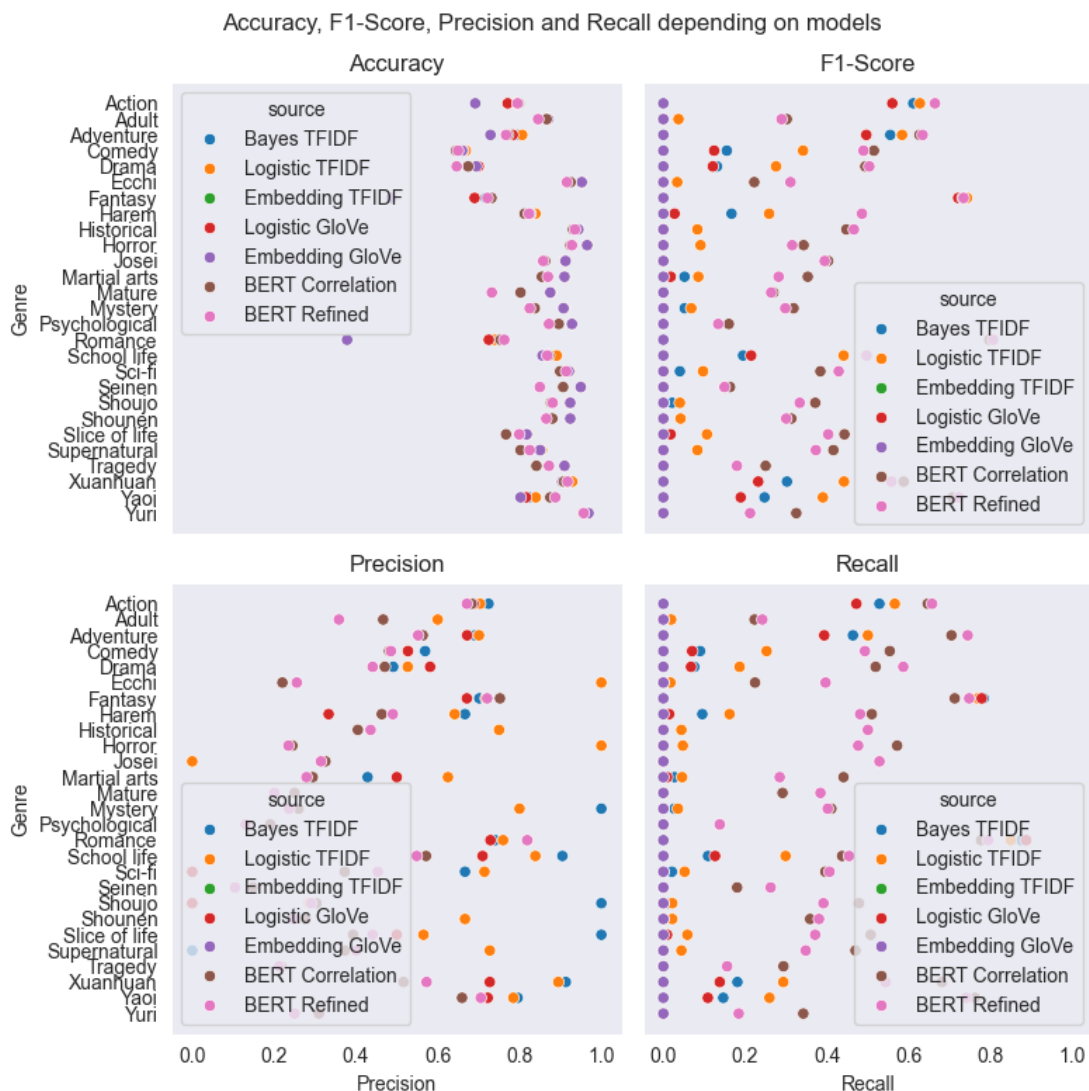
figure, axes = plt.subplots(2, 2, sharey=True, figsize=(8,8), sharex=True)
figure.suptitle('Accuracy, F1-Score, Precision and Recall depending on models')

# Scatterplot for all the metrics
sns.scatterplot(labelMetrics, y = "Genre", x="Accuracy", hue="source",
               ↪ax=axes[0,0])
sns.scatterplot(labelMetrics, y = "Genre", x="F1-Score", hue="source",
               ↪ax=axes[0,1])
sns.scatterplot(labelMetrics, y = "Genre", x="Precision", hue="source",
               ↪ax=axes[1,0])
sns.scatterplot(labelMetrics, y = "Genre", x="Recall", hue="source",
               ↪ax=axes[1,1])

# Set the titles
axes[0,0].set_title("Accuracy")
axes[0,1].set_title("F1-Score")
axes[1,0].set_title("Precision")
axes[1,1].set_title("Recall")

plt.tight_layout()
plt.show()

```



Now, what can be said about these?

Firstly, the median of the Hamming similarity is almost always the same and does not seem significant. As mentioned earlier, the Hamming similarity is very sensitive to the size of the label space, and since we generally have a limited number of labels, this metric favours the large number of zeros in our vector, which is not ideal for predicting rare labels.

Secondly, the accuracy product is always low. This is normal because it actually corresponds to the probability of a perfect match, which is not probable with 27 labels. Nevertheless, we observe that our neural models, which we developed to improve rare label prediction, have a lower chance of achieving a perfect match. Again, this is logical as we encourage the inference of rare labels, which reduces the likelihood of a perfect match in the event of a mistake.

Thirdly, the median Jaccard similarity is decent overall, except for the embedding similarity, where the median Jaccard similarity is 0. As we saw earlier, the results with this predictor were basically

the highest frequencies. Our Neural models still achieve the best results, although Bayes and Logistics also produce decent results.

Ultimately, we would like to determine whether there are significant differences in label inferences or if our neural models are simply enhanced Bayes/logistic classifiers.

To achieve this, we can examine the other metrics per label.

Regarding the accuracy of the labels, we can see that the document embedding similarity approach (GloVe and others) generally leads to the best accuracy because its classification is similar to the “most frequent label” classifier. This also means something important: since the accuracy is high, the issue is no longer accuracy, but rather the correct classification of label combinations and rare labels. In our case, we are most interested in precision, recall and F1 score, which are more resistant to label imbalance. Regarding whether neural approaches are better in terms of accuracy when disregarding embedding similarities, we do not observe significant differences compared to other methods.

From an F1-score perspective, the results are generally mediocre, with scores below 50% most of the time. However, we observe significant differences between our baselines and the neural models. While the results are comparable for frequent labels such as **Fantasy** and **Romance**, the neural models clearly outperform the baselines for genres such as **Shoujo**, **Supernatural**, **Sci-Fi**, **Yaoi** and **Historical**, which are representative of rarer or more complex genres (e.g. confusion between **Romance** and **Yaoi**, or **Supernatural** and **Fantasy**). This is good news because it suggests that we were able to discriminate between genres more easily and resolve ambiguity between related ones. As for whether there is a difference between the two neural models, we do not strictly observe an improvement from one to the other based on the F1 score alone. But is the reason why the F1 scores are similar between the two models the same? Let us examine Precision and Recall.

The results for precision vary considerably depending on the labels. Firstly, the baselines often achieve the best precision. This is interesting because it means that our baseline tends to correctly classify positives, whereas our neural models struggle with this. However, as mentioned earlier, some genres have very poor precision, such as **Yaoi**, **Supernatural**, **Shojo** and **Josei**, while others have strong precision: ‘**Romance**’, **Fantasy** and **Action**. Ultimately, these results can probably be attributed to the semantic correlation revealed by these genres. Regarding the differences between the two neural models, the *BERT Corr* model achieves slightly better precision overall, although it does not display particular advantages in terms of genre.

The story is very different when it comes to recall. Traditional approaches tend to have recall rates closer to zero (e.g. **Adult**, **Ecchi**, **Harem**, **Historical**, **Horror**), while neural approaches dominate all categories except **Fantasy** and **Romance**. The gap in recall rate is minimal for the category **Action** (approximately 0.08 points), and the largest gap is over 0.5 points. This is interesting because it truly shows that our neural models capture the positives better than our baseline does. In other words, it shows that imbalance and correlation between labels should be taken into account when making inferences. As for whether a particular neural method is better for recall, we observe that none is truly better than the other. Nonetheless, the results differ significantly, indicating that higher-order correlation efficiencies depend heavily on the embedding tested. It would be interesting to identify the cases where one method is better than the others.

Now, we will examine the worst classification error made by our baselines and establish whether switching to neural models has been beneficial.

(The id were: 324, 325, 340, 344, 354, 799, 803, 809, 821, 829, 830, 832, 839, 846, 1029, 1032, 1033,

1202, 1209)

```
[62]: # Get the entries where the errors were analysed
errorsToCheck = [324, 325, 340, 344, 354, 799, 803, 809, 821, 829, 830, 832,
↪839, 846, 1029, 1032, 1033, 1202, 1209]

pd.DataFrame.join(
    # Get the results for the baselines
    pd.DataFrame.join(
        rowMetricsBayesTFIDF.loc[errorsToCheck, ["Description", "Genres",
↪"Predicted Genres", "Jaccard"]],
        rowMetricsLogisticTFIDF.loc[errorsToCheck, ["Predicted Genres",
↪"Jaccard"]],
        lsuffix=" BayesTFIDF", rsuffix=" LogisticTFIDF"
    ),
    # Get the results for the two neural models
    pd.DataFrame.join(
        rowMetricsNeuralBaseline.loc[errorsToCheck, ["Predicted Genres",
↪"Jaccard"]],
        rowMetricsNeuralRefined.loc[errorsToCheck, ["Predicted Genres",
↪"Jaccard"]],
        lsuffix=" BERTCorr", rsuffix=" BERTRefined"
    )
    # Get only the columns of interest
)[["Description", "Genres",
    "Predicted Genres BayesTFIDF", "Predicted Genres LogisticTFIDF",
↪"Predicted Genres BERTCorr", "Predicted Genres BERTRefined",
    "Jaccard BayesTFIDF", "Jaccard LogisticTFIDF", "Jaccard BERTCorr",
↪"Jaccard BERTRefined"]]
```

```
[62]:                                     Description \
324   He is the eldest grandson of the Mu clan who h...
325   The entire world bullies you, hates you, and ...
340   Her step-sister just died three months ago. No...
344   Li Wei, who was hit by a speeding truck, came ...
354   "I don't want to wash the dishes today."\n"I'l...
799   Song Luan transmigrated into a book called "Th...
803   As a person who transmigrated into a novel wri...
809   Peach farmer Toru discovers a sweet-smelling, ...
821   Back in 1996, the eldest daughter of Oldman Xi...
829   This book consists of a string of short storie...
830   The story of searching for a Tabi Pouch to put...
832   The distance of a perfect stand-in is the dist...
839   Bill: It referred to the devil. It was also th...
846   Hu Yu chased Fu Anli for three years. After be...
1029  B-class adventurer Gray found an orphan who so...
1032  20XX. Japan has become a super-aging society w...
```

1033 Four years ago, a mysterious air crash took aw...
 1202 On April 1st, 20XX, I came across something go...
 1209 One day, a virus capsule fell into a region an...

	Genres \
324	[Adult, Josei, Mature, Romance]
325	[Adventure, Drama, Horror, Romance, Supernatur...
340	[Drama, Horror, Tragedy]
344	[Action, Adventure, Fantasy, Harem, Mystery, P...
354	[Drama, Romance, Tragedy, Yaoi]
799	[Adult, Historical, Josei, Supernatural, Tragedy]
803	[Adventure, Drama, Martial arts, Mature, Psych...
809	[Fantasy, Yaoi]
821	[Drama, Fantasy]
829	[Slice of life]
830	[Drama, Tragedy]
832	[Drama, Yaoi]
839	[Drama, Yaoi]
846	[Adult, Comedy, Fantasy, School life, Supernat...
1029	[Action, Adventure, Drama, Slice of life]
1032	[Action, Horror, School life, Shounen, Tragedy]
1033	[Mystery, Yaoi]
1202	[Comedy, Supernatural, Xuanhuan, Yaoi]
1209	[Drama, Mystery, Tragedy]

	Predicted Genres BayesTFIDF	Predicted Genres LogisticTFIDF \
324	[Romance]	[]
325	[Romance]	[Action]
340	[Romance]	[Fantasy, Romance]
344	[Romance]	[Romance]
354	[Fantasy, Romance]	[]
799	[Fantasy, Romance]	[Fantasy, Romance]
803	[Romance]	[Comedy, Romance, Yaoi]
809	[Romance]	[Romance]
821	[Romance]	[Comedy, Romance]
829	[Fantasy, Romance]	[]
830	[Fantasy, Romance]	[Fantasy, Romance]
832	[Romance]	[]
839	[Romance]	[Romance]
846	[Romance]	[Romance]
1029	[Fantasy, Romance]	[Fantasy, Romance]
1032	[Fantasy, Romance]	[Comedy, Romance]
1033	[Action]	[]
1202	[Fantasy, Romance]	[Adventure]
1209	[Action, Fantasy]	[Action]

Predicted Genres BERTCorr \

324 [Historical, Josei, Mature, Romance]
 325 [Comedy, Drama, Horror, Mystery, Romance, Supe...
 340 [Drama, Josei, Mystery, Romance]
 344 [Adventure, Comedy, Drama, Fantasy, Mystery, R...
 354 [Fantasy, Romance]
 799 [Comedy, Fantasy, Historical, Josei, Romance]
 803 [Comedy, Mystery, Romance, Supernatural, Yaoi]
 809 [Drama, Romance, Slice of life, Yaoi]
 821 [Drama, Josei, Romance, Slice of life]
 829 [Mature, Mystery, School life, Slice of life, ...
 830 [Horror, Mystery, Slice of life, Supernatural,...
 832 [Drama, Romance, Yaoi]
 839 [Drama, Romance, Tragedy]
 846 [Comedy, Romance, Yaoi]
 1029 [Adventure, Fantasy, Tragedy, Yaoi]
 1032 [Horror]
 1033 [Action, Adventure, Martial arts, Mystery, Sei...
 1202 [Adventure, Comedy, Supernatural, Xuanhuan]
 1209 [Action, Adventure, Fantasy, Horror, Mystery, ...

	Predicted Genres BERTRefined	Jaccard BayesTFIDF \
324	[Drama, Historical, Josei, Martial arts, Matur...	0.250000
325	[Comedy, Drama, Mystery, Romance, Supernatural...	0.166667
340	[Drama, Josei, Romance]	0.000000
344	[Comedy, Drama, Fantasy, Mystery, Psychological...	0.000000
354	[Fantasy, Yaoi]	0.200000
799	[Fantasy, Historical, Josei, Romance]	0.000000
803	[Comedy, Mystery, Romance, Supernatural, Yaoi]	0.000000
809	[Drama, Romance, Slice of life, Yaoi]	0.000000
821	[Drama, Josei, Romance]	0.000000
829	[Ecchi, Harem, Mature, Mystery, Psychological,...	0.000000
830	[Supernatural, Yaoi]	0.000000
832	[Drama, Romance, Yaoi]	0.000000
839	[Drama, Historical, Mature, Mystery, Romance, ...	0.000000
846	[Comedy, Romance, Yaoi]	0.000000
1029	[Fantasy, Yaoi]	0.000000
1032	[Drama, Fantasy, Supernatural]	0.000000
1033	[Action, Adventure, Drama, Fantasy, Mystery, S...	0.000000
1202	[Adventure, Xuanhuan]	0.000000
1209	[Action, Adventure, Fantasy, Horror, Mystery, ...	0.000000

	Jaccard LogisticTFIDF	Jaccard BERTCorr	Jaccard BERTRefined
324	0.000	0.600000	0.428571
325	0.000	0.625000	0.500000
340	0.000	0.166667	0.200000
344	0.000	0.250000	0.300000
354	0.000	0.200000	0.200000

799	0.000	0.250000	0.285714
803	0.125	0.100000	0.100000
809	0.000	0.200000	0.200000
821	0.000	0.200000	0.250000
829	0.000	0.200000	0.000000
830	0.000	0.000000	0.000000
832	0.000	0.666667	0.666667
839	0.000	0.250000	0.285714
846	0.000	0.285714	0.285714
1029	0.000	0.142857	0.000000
1032	0.000	0.200000	0.000000
1033	0.000	0.166667	0.142857
1202	0.000	0.600000	0.200000
1209	0.000	0.222222	0.100000

Now that we have these results, what can we conclude?

Firstly, except for entry *803*, the classifications made by both models were strictly better than those made by our baseline in every case. Secondly, *BERTCorr* was generally better than *BERTRefined*. Thirdly, the neural model tends to infer more labels than our baseline (probably because we took label correlations into consideration).

Now, what about the **Romance/Yaoi** (803, 839, 846) and the **Supernatural/Fantasy** (799) confusions we made with our baselines? - [803]: For this one, the confusion between **Yaoi** and **Romance** is still clear, but contrary to Bayes, which put only one of those, the Neural Models took into account the correlation and put both. This warrants at least something to be true and therefore have a Jaccard Index over 0. Still, Logistic is better here as it has the tendency to put less labels. - [839]: Here, we observe something interesting, despite *BERTCorr* predicting only half of the real answer, *BERTRefined* did produce a superset of the real answer. When we check the additional labels, we also observe that they make sense; indeed, the words *devil*, *underground*, *memory*, and *secret* can clearly be associated to **Tragedy**, **Mature**, or **Mystery**. In the end, the neural models would have inferred something closer to what we, humans, could have inferred by reading the description. - [799]: As observed in the newly inferred labels, the confusion for **Supernatural/Fantasy** was not solved. Still, the results are interesting because the inferred labels are close to what a human would have done. The **Josei** tag was correctly inferred, as well as the **Romance** tag, which could have been correct by reading the description. - [846]: Here, the same remarks as what has been told for (803) can be done.

What about descriptions that use implicit knowledge (324), short descriptions (354) or descriptions of staff or materials instead of the story itself (829)?

- [324]: Now, the results are very interesting. In this example, the baselines were not able to correctly infer the ‘I got rescued by a saviour, I fell in love directly after’; but the neural models did. In addition to just infer **Romance**, as Bayes did, they also correctly inferred **Josei** and **Mature** (but not **Adult**). Reading the description, **Historical**, **Drama**, and **Martial Art** were also plausible, so it is not really a problem to predict them. This example shows that neural models can capture some implicit knowledge through common tropes (probably thanks to the contextual embedding of BERT).
-
-

In the end, we demonstrated that both neural models can mitigate the **Romance/Yaoi** confusion without eliminating it completely. They can capture implicit knowledge, but this is still limited to the description type (e.g. length, and whether or not it talks about the story).

The only question remaining is: where does the neural models fails? Let's first begin with *BERT-Corr*.

```
[63]: rowMetricsNeuralBaseline.tail(5)
```

```
[63]:
```

	Description \			
1060		N/A\n		
96	Three years ago, the sky over the imperial cap...			
1061	It's enough that my hands and legs are bound s...			
870	A short story about a young wolf and tombstone\n			
374		N/A\n		

	Genres \			
1060	[Ecchi, Fantasy, Martial arts, Mature]			
96	[Drama, Historical, Yaoi]			
1061	[Mystery, Supernatural]			
870	[Fantasy, Tragedy]			
374	[Seinen]			

	Predicted Genres	Jaccard	Hamming
1060	[Horror]	0.0	0.814815
96	[Action, Martial arts, Mystery, Tragedy, Xuanh...	0.0	0.703704
1061	[Comedy, School life]	0.0	0.851852
870	[Supernatural, Yaoi]	0.0	0.851852
374	[Horror]	0.0	0.925926

Observing the descriptions, when it is empty (*N/A*), the inferred genre is **Horror** (why not, it is indeed terrifying!). But in those cases, it is clearly our fault; we should have detected this way earlier. Let us check very quickly if this is disastrous or not (if half the dataset descriptions are empty or not).

```
[64]: print(f"The proportion of the dataset being empty is: {(X[:,0,0]=='N/A\n').
↪sum()/X.shape[0]*100:.3f}%")
```

The proportion of the dataset being empty is: 0.561%

This result is reasonable and should not destroy our results and observations. Now, back to the other description:

- [96]: In this description, the format is quite special, and more shaping a poem than a description, making it very original. From it, *BayseCorr* made good guesses (I would probably have inferred something similar), but we can see a confusion appearing between **Historical** and **Martial Arts/Xuanhuan**, which are frequently inferred together.
- [1061]: Even though I tend to be formal in reports, excuse me for this, but WTF is this: the Joker (DC Comics) and Sherlock Holmes being in a Mental Hospital managed by the [SCP Foundation](#) (weird meme of the internet). Obviously, it has to be **Comedy**, right? Unfortun-

nately for the models, the real labels are **Mystery** and **Supernatural** (which seems right for the SCP Foundation). To infer these labels, we have to know about the SCP Foundation, which is itself hidden as an inside joke (meme) of the internet. It is not surprising that our models cannot infer these without this information.

And what about the other version, *BERTRefined*?

```
[65]: rowMetricsNeuralRefined.tail(5)
```

```
[65]:
```

	Description \	Genres \	Predicted Genres	Jaccard	Hamming
1159	Xu Qinghe was attacked while investigating a s...	[Adult, Tragedy, Yuri]	[Drama]	0.0	0.851852
823	A tales of a girl and a certain mother cat\n	[Fantasy, Slice of life]	[Historical, Romance, Shoujo]	0.0	0.814815
66	How does an unmarried woman suddenly have an a...	[Comedy, Fantasy, Shoujo, Slice of life]	[Drama, Horror, Mystery, Psychological, Romanc...	0.0	0.592593
1089	Because of his master's wine, he had somehow b...	[Harem, Martial arts, Romance]	[Fantasy, Xuanhuan, Yaoi]	0.0	0.777778
1073	One day Xiao Jian discovers his neighbor Uncle...	[Adult, Mature]	[Josei, Romance]	0.0	0.851852

Here, *BERTRefined* provides examples of failures on short descriptions (823), confusion between **Drama** and **Tragedy** (1159), descriptions that are very long but does not truly fit their attributed genres (66), confusion between **Yaoi/Adult/Romance**, as well as **Martial Arts/Xuanhuan** (1089), and another confusion between **Adult/Mature/Romance** and **Josei** (1073, do not read the description, it is awful).

Ultimately, we cannot tell which neural model is better, nor can we identify the specific differences between them. The only thing we can say is that, despite their visible differences in terms of inferences, they all fail in the same ways: empty or short descriptions, awkward labelling, a particular format, production descriptions instead of story descriptions, and descriptions that are very convoluted or require particular knowledge or references to understand the mapping to their genres.

Apart from the potential biases mentioned at the beginning of the report and the various confusions observed throughout, we did not detect any particularly strong biases for these classifiers. However, Bayes introduced one by labelling everything as either ‘Romance’ or **Fantasy**.

1.9 Conclusion, Area for improvement and Academic Integrity Declaration

In this work, we explored the *Web/Light Novel Dataset* and used various tools, including Bayes, Logistic, GloVe and BERT-based methods, to predict the genre of a web/light novel from its description. To achieve this, we extensively studied the links between the labels and considered how these could be leveraged in the context of a highly imbalanced multi-labelling task. Additionally, we reflected on evaluation metrics for multilabel classification, ranging from classic accuracy to the Jaccard index, to transition from perfect matches to a more appropriate evaluation.

Along the way, we identified two main approaches to multilabel classification: Binary relevance (one classifier per label), which is the simplest approach but cannot handle label correlations, and neural networks, which leverage label correlations for better classification.

Ultimately, we were unable to produce a model that matched perfectly with high accuracy, but we did develop somewhat usable and satisfactory models that were free from obvious biases: *BERT Refined* and *BERT Correlation*. While far from perfect, these models produced convincing results similar to those that could be achieved by a human in a limited context involving issues such as incorrect labelling, short descriptions, inside jokes and production descriptions.

Concerning **area of improvements**, we clearly see four possible ways: 1. Binary relevance over Bayes or logistic regression does not take correlation into account. It would be interesting to see if inferring fuzzy association rules between labels could solve this issue. This would mean that labels would no longer be either fully true or false, but could take values from 0 (fully false) to 1 (fully true). While this hypothesis is interesting because it is closer to reality, we would also need to find a fuzzification technique (probably rule-based) to achieve this, which could be quite challenging.

2. In this report, our focus was on genres, but the dataset actually contains tags, which are essentially subgenres. It could therefore be interesting to see if leveraging these tags could help us infer the labels in some way. However, this could be challenging given the 891 additional subgenres, each with their own imbalances and correlations.
3. Similarly, we could see if adding the novel’s title to the classification problem would make a difference, particularly in the WN/LN industry where titles that describe the plot, such as “*I Was Reincarnated as the 7th Prince so I Can Take My Time Perfecting My Magical Ability*” or “*Is It Wrong to Try to Pick Up Girls in a Dungeon?*”, are very common.
4. We observed a lot of confusion between the tags. In this respect, we could try merging genres into higher families more thoroughly, or take this confusion into account in the evaluation. For example, we could say: “If I infer **Fantasy** instead of **Supernatural**, it is as if I had inferred **Supernatural** with 70% certainty.” (Fuzzy evaluation). This would be an interesting approach and would probably be the best way to represent non-semantically mergeable overlapping genres (e.g. **Adult!**=**Romance**).

Academic Integrity Declaration *The tools used for this project are:* - Python and libraries PyTorch, SciKit-Learn, NumPy, Pandas and so on. - Particular websites: Documentation, Stack Overflow, Data Science Stack Exchange, Mathematics Stack Exchange, Wikipédia, Web Archive (for SciKit Multilearn), GeeksForGeeks, Google Scholar, GitHub, Medium, and diverse others. - AI tools: Mistral and ChatGPT free version without connections, DeepL.

How was the code produced? It was produced entirely by hand in VSCode, without the use of external tools or chatbots, unless this was explicitly acknowledged. If acknowledged, external coding tools (chatbots) were used for specific non-technical tasks, such as producing a particular

legend format for Matplotlib or associating genres with colours.

Chatbot usage: Two chatbots were used for debugging purposes only (unless specified): the free version of Mistral and ChatGPT without identification. This debugging included resolving Spacy/PyTorch incompatibilities, unrecognised CUDA/GPU issues, and DLL Windows errors. All information provided by the chatbots was verified manually by cross-referencing multiple sources (documentation, papers, blogs, etc.) and through experimentation (checking coherence and applicability).

For the sake of academic integrity, no codes were copied and pasted without acknowledgement. In addition, all analysis and explanations (in code comments or in the report) were written by hand and are not based on any outputs from chatbots (except for the use of DeepL Translate/Write to recall an expression or word, reformulate sentences, and check grammar and spelling).

If you have any doubt about this declaration, feel free to contact Mr. LESAGE.

Arno LESAGE no. 22202985

Minh QUAN HOANG no. 22509118